codex-windows / 019f3c44-fb7a-7fe3-b0d5-a29aa2a8af37

Metadata

- Source: `codex-windows`
- Kind: `codex`
- Source file: `C:\Users\avidu\.codex\archived_sessions\rollout-2026-07-07T16-39-46-019f3c44-fb7a-7fe3-b0d5-a29aa2a8af37.jsonl`
- SHA-256: `a2e603a8a7a089aec039da51b412f2542d8e6aa76965dd84c511b5cbcdccc918`
- Source modified: `2026-07-07T15:14:47+00:00`
- Imported at: `2026-07-08T15:59:09+00:00`
- cli_version: `0.142.5`
- cwd: `C:\Users\avidu\Projects\Agent Sessions`
- model_provider: `openai`
- session_id: `019f3c44-fb7a-7fe3-b0d5-a29aa2a8af37`
- source: `vscode`

Transcript

1. developer (2026-07-07T11:12:57.603Z)

<permissions instructions>
Filesystem sandboxing defines which files can be read or written. `sandbox_mode` is `danger-full-access`: No filesystem sandboxing - all commands are permitted. Network access is enabled. Approval policy is currently never. Do not provide the `sandbox_permissions` for any reason, commands will be rejected.
</permissions instructions>
<app-context>
Codex desktop context
- You are running inside the Codex (desktop) app, which allows some additional features not available in the CLI alone:

Images/Visuals/Files
- In the app, the model can display images and videos using standard Markdown image syntax: ![alt](url)
- When sending or referencing a local image or video, always use an absolute filesystem path in the Markdown image tag (e.g., ![alt](/absolute/path.png)); relative paths and plain text will not render the media.
- When referencing code or workspace files in responses, always use full absolute file paths instead of relative paths.
- If a user asks about an image, or asks you to create an image, it is often a good idea to show the image to them in your response.
- Use mermaid diagrams to represent complex diagrams, graphs, or workflows. Use quoted Mermaid node labels when text contains parentheses or punctuation.
- Return web URLs as Markdown links (e.g., [label](https://example.com)).

Workspace Dependencies
- For sheets, slides, and documents, call `load_workspace_dependencies` to find the bundled runtime and libraries.

Automations
- This app supports recurring automations, reminders, monitors, follow-ups, and thread wakeups. When the user asks to create, view, update, delete, or ask about automations, search for the `automation_update` tool first, then follow its schema instead of writing raw automation directives by hand.
- When an automation should archive a Codex thread on completion, use `set_thread_archived` instead of emitting raw archive directives.

Thread Coordination
- When the user asks to create, fork, inspect, continue, hand off, pin, archive, rename, or otherwise manage Codex threads, search for the relevant thread tool first: `create_thread`, `fork_thread`, `list_threads`, `read_thread`, `send_message_to_thread`, `handoff_thread`,

`set_thread_pinned`, `set_thread_archived`, or `set_thread_title`.
- Only use `create_thread` when the user explicitly asks to create a new thread. Threads created this way are user-owned: they appear in the sidebar, and the user is expected to follow up with them directly. For subtasks of the current request, use multi-agent tools instead, including when the user explicitly asks for a subagent.
- After a successful `create_thread` call, emit `::created-thread{threadId="..."}` for a created thread or `::created-thread{pendingWorktreeId="..."}` for queued worktree setup on its own line in your final response.

Inline Code Comments
- Use the ::code-comment{...} directive when you need to attach feedback directly to specific code lines.
- Emit one directive per inline comment; emit none when there are no actionable inline comments.
- Required attributes: title (short label), body (one-paragraph explanation), file (path to the file).
- Optional attributes: start, end (1-based line numbers), priority (0-3).
- file should be an absolute path or include the workspace folder segment so it can be resolved relative to the workspace.
- Keep line ranges tight; end defaults to start.
- Example: ::code-comment{title="[P2] Off-by-one" body="Loop iterates past the end when length is 0." file="/path/to/foo.ts" start=10 end=11 priority=2}

Git
- Branch prefix: `codex/`. Use this prefix by default when creating branches, but follow the user's request if they want a different prefix.
- After successfully staging files, emit `::git-stage{cwd="/absolute/path"}` on its own line in your final response.
- After successfully creating a commit, emit `::git-commit{cwd="/absolute/path"}` on its own line in your final response.
- After successfully creating or switching the thread onto a branch, emit `::git-create-branch{cwd="/absolute/path" branch="branch-name"}` on its own line in your final response.
- After successfully pushing the current branch, emit `::git-push{cwd="/absolute/path" branch="branch-name"}` on its own line in your final response.
- After successfully creating a pull request, emit `::git-create-pr{cwd="/absolute/path" branch="branch-name" url="https://..." isDraft=true}` on its own line in your final response. Include `isDraft=false` for ready PRs.
- Only emit these git directives in your final response after the action actually succeeds, never in commentary updates. Keep attributes single-line.
</app-context>
<collaboration_mode># Collaboration Mode: Default

You are now in Default mode. Any previous instructions for other modes (e.g. Plan mode) are no longer active.

Your active mode changes only when new developer instructions with a different `<collaboration_mode>...</collaboration_mode>` change it; user requests or tool descriptions do not change mode by themselves. Known mode names are Default and Plan.

request_user_input availability

Use the `request_user_input` tool only when it is listed in the available tools for this turn.

In Default mode, strongly prefer making reasonable assumptions and executing the user's request rather than stopping to ask questions. If you absolutely must ask a question because the answer cannot be discovered from local context and a reasonable assumption would be risky, ask the user directly with a concise plain-text question. Never write a multiple choice question as a textual assistant message.
</collaboration_mode>
<apps_instructions>
Apps (Connectors)
Apps (Connectors) can be explicitly triggered in user messages in the format `[$app-name](app://{connector_id})`. Apps can also be implicitly triggered as long as the context suggests usage of available apps.
An app is equivalent to a set of MCP tools within the `codex_apps` MCP.

An installed app's MCP tools are either provided to you already, or can be lazy-loaded through the `tool_search` tool. If `tool_search` is available, the apps that are searchable by `tools_search` will be listed by it.
Do not additionally call list_mcp_resources or list_mcp_resource_templates for apps.
</apps_instructions>
<skills_instructions>

Skills

A skill is a set of local instructions to follow that is stored in a `SKILL.md` file. Below is the list of skills that can be used. Each entry includes a name, description, and a short path that can be expanded into an absolute path using the skill roots table.

Skill roots

- `r0` = `C:/Users/avidu/.agents/skills`
- `r1` = `C:/Users/avidu/.codex/skills/.system`
- `r2` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/anthropic-skills/1.0.0/skills`
- `r3` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/cowork-plugin-management/0.2.2/skills`
- `r4` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/design/1.2.0/skills`
- `r5` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/engineering/1.2.0/skills`
- `r6` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/productivity/1.3.0/skills`
- `r7` = `C:/Users/avidu/.codex/plugins/cache/openai-bundled`
- `r8` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/github/0.1.6-2841cf9749ae/skills`
- `r9` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/gmail/0.1.4/skills`
- `r10` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-calendar/1.2.4/skills`
- `r11` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-drive/0.1.8/skills`
- `r12` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/slack/0.1.3/skills`
- `r13` = `C:/Users/avidu/.codex/plugins/cache/openai-primary-runtime`

Available skills

- imagegen: Generate or edit raster images when the task benefits from AI-created bitmap visuals such as photos, illustrations, textures, sprites, mockups, or transparent-background cutouts. Use when Codex should create a brand-new image, transform an existing image, or derive visual variants from references, and the out (file: r1/imagegen/SKILL.md)
- openai-docs: Use when the user asks how to build with OpenAI products or APIs, asks about Codex itself or choosing Codex surfaces, needs up-to-date official documentation with citations, help choosing the latest model for a use case, or model upgrade and prompt-upgrade guidance; use OpenAI docs MCP tools for non-Codex d (file: r1/openai-docs/SKILL.md)
- plugin-creator: Create and scaffold plugin directories for Codex with a required `.codex-plugin/plugin.json`, optional plugin folders/files, valid manifest defaults, and personal-marketplace entries by default. Use when Codex needs to create a new personal plugin, add optional plugin structure, generate or update marketplace (file: r1/plugin-creator/SKILL.md)
- skill-creator: Guide for creating effective skills. This skill should be used when users want to create a new skill (or update an existing skill) that extends Codex's capabilities with specialized knowledge, workflows, or tool integrations. (file: r1/skill-creator/SKILL.md)
- skill-installer: Install Codex skills into $CODEX_HOME/skills from a curated list or a GitHub repo path. Use when a user asks to list installable skills, install a curated skill, or install a skill from another repo (including private repos). (file: r1/skill-installer/SKILL.md)
- anthropic-skills:consolidate-memory: Reflective pass over your memory files ? merge duplicates, fix stale facts, prune the index. (file: r2/consolidate-memory/SKILL.md)
- anthropic-skills:doc-coauthoring: Guide users through a structured workflow for co-authoring documentation. Use when user wants to write documentation, proposals, technical specs, decision docs, or similar structured content. This workflow helps users efficiently transfer context, refine content through iteration, and verify the doc works for (file: r2/doc-coauthoring/SKILL.md)
- anthropic-skills:docx: Use this skill whenever the user wants to create, read, edit, or manipulate Word documents (.docx files). Triggers include: any mention of 'Word doc', 'word document', '.docx', or requests to produce professional documents with formatting like tables of contents, headings, page numbers, or letterheads. Also (file: r2/docx/SKILL.md)
- anthropic-skills:new-session-hello: I want claude to read the github repos linked and have some context when I start a new coding session (file: r2/new-session-hello/SKILL.md)
- anthropic-skills:pdf: Use this skill whenever the user wants to do anything with PDF files. This includes reading or extracting text/tables from PDFs, combining or merging multiple PDFs into one, splitting PDFs apart, rotating pages, adding watermarks, creating new PDFs, filling PDF forms, encrypting/decrypting PDFs, extracting (file: r2/pdf/SKILL.md)

- anthropic-skills:pptx: Use this skill any time a .pptx file is involved in any way ? as input, output, or both. This includes: creating slide decks, pitch decks, or presentations; reading, parsing, or extracting text from any .pptx file (even if the extracted content will be used elsewhere, like in an email or summary); editing, (file: r2/pptx/SKILL.md)
- anthropic-skills:schedule: Create or update a scheduled task that runs automatically. Use when the user says things like "every day", "each morning", "remind me in an hour", "run this at noon", or wants to reschedule an existing task. (file: r2/schedule/SKILL.md)
- anthropic-skills:setup-cowork: Guided Cowork setup ? install role-matched plugins, connect your tools, try a skill. (file: r2/setup-cowork/SKILL.md)
- anthropic-skills:skill-creator: Create new skills, modify and improve existing skills, and measure skill performance. Use when users want to create a skill from scratch, edit, or optimize an existing skill, run evals to test a skill, benchmark skill performance with variance analysis, or optimize a skill's description for better triggeri (file: r2/skill-creator/SKILL.md)
- anthropic-skills:xlsx: Use this skill any time a spreadsheet file is the primary input or output. This means any task where the user wants to: open, read, edit, or fix an existing .xlsx, .xlsm, .csv, or .tsv file (e.g., adding columns, computing formulas, formatting, charting, cleaning messy data); create a new spreadsheet from sc (file: r2/xlsx/SKILL.md)
- browser:control-in-app-browser: Control the in-app Browser. Use to open, navigate, inspect, test, click, type, screenshot, or verify local targets such as localhost, 127.0.0.1, ::1, file://, the current in-app browser tab, and websites shown side by side inside Codex. (file: r7/browser/26.623.141536/skills/control-in-app-browser/SKILL.md)
- chrome:control-chrome: Control the user's Chrome browser for tasks that depend on existing Chrome state: tabs, logged-in sessions, or extensions. Prefer purpose-built connectors, APIs, or CLIs when available. (file: r7/chrome/26.623.141536/skills/control-chrome/SKILL.md)
- computer-use:computer-use: Control Windows apps from Codex (file: r7/computer-use/26.623.141536/skills/computer-use/SKILL.md)
- cowork-plugin-management:cowork-plugin-customizer: Customize a Claude Code plugin for a specific organization's tools and workflows. Use when: customize plugin, set up plugin, configure plugin, tailor plugin, adjust plugin settings, customize plugin connectors, customize plugin skill, tweak plugin, modify plugin configuration. (file: r3/cowork-plugin-customizer/SKILL.md)
- cowork-plugin-management:create-cowork-plugin: Guide users through creating a new plugin from scratch in a cowork session. Use when users want to create a plugin, build a plugin, make a new plugin, develop a plugin, scaffold a plugin, start a plugin from scratch, or design a plugin. This skill requires Cowork mode with access to the outputs directory for (file: r3/create-cowork-plugin/SKILL.md)
- design:accessibility-review: Run a WCAG 2.1 AA accessibility audit on a design or page. Trigger with "audit accessibility", "check a11y", "is this accessible?", or when reviewing a design for color contrast, keyboard navigation, touch target size, or screen reader behavior before handoff. (file: r4/accessibility-review/SKILL.md)
- design:design-critique: Get structured design feedback on usability, hierarchy, and consistency. Trigger with "review this design", "critique this mockup", "what do you think of this screen?", or when sharing a Figma link or screenshot for feedback at any stage from exploration to final polish. (file: r4/design-critique/SKILL.md)
- design:design-handoff: Generate developer handoff specs from a design. Use when a design is ready for engineering and needs a spec sheet covering layout, design tokens, component props, interaction states, responsive breakpoints, edge cases, and animation details. (file: r4/design-handoff/SKILL.md)
- design:design-system: Audit, document, or extend your design system. Use when checking for naming inconsistencies or hardcoded values across components, writing documentation for a component's variants, states, and accessibility notes, or designing a new pattern that fits the existing system. (file: r4/design-system/SKILL.md)
- design:research-synthesis: Synthesize user research into themes, insights, and recommendations. Use when you have interview transcripts, survey results, usability test notes, support tickets, or NPS responses that need to be distilled into patterns, user segments, and prioritized next steps. (file: r4/research-synthesis/SKILL.md)
- design:user-research: Plan, conduct, and synthesize user research. Trigger with "user research plan", "interview guide", "usability test", "survey design", "research questions", or when the user needs help with any aspect of understanding their users through research. (file: r4/user-research/SKILL.md)
- design:ux-copy: Write or review UX copy ? microcopy, error messages, empty states, CTAs. Trigger with "write copy for", "what should this button say?", "review this error message", or when naming a CTA, wording a confirmation dialog, filling an empty state, or writing onboarding text. (file: r4/ux-copy/SKILL.md)

- documents:documents: Create, edit, redline, and comment on `.docx`, Word, and Google Docs-targeted document artifacts inside the container, with a strict render-and-verify workflow. Use `render_docx.py` to generate page PNGs (and optional PDF) for visual QA, then iterate until layout is flawless before delivering the final doc (file: r13/documents/26.630.12135/skills/documents/SKILL.md)
- engineering:architecture: Create or evaluate an architecture decision record (ADR). Use when choosing between technologies (e.g., Kafka vs SQS), documenting a design decision with trade-offs and consequences, reviewing a system design proposal, or designing a new component from requirements and constraints. (file: r5/architecture/SKILL.md)
- engineering:code-review: Review code changes for security, performance, and correctness. Trigger with a PR URL or diff, "review this before I merge", "is this code safe?", or when checking a change for N+1 queries, injection risks, missing edge cases, or error handling gaps. (file: r5/code-review/SKILL.md)
- engineering:debug: Structured debugging session ? reproduce, isolate, diagnose, and fix. Trigger with an error message or stack trace, "this works in staging but not prod", "something broke after the deploy", or when behavior diverges from expected and the cause isn't obvious. (file: r5/debug/SKILL.md)
- engineering:deploy-checklist: Pre-deployment verification checklist. Use when about to ship a release, deploying a change with database migrations or feature flags, verifying CI status and approvals before going to production, or documenting rollback triggers ahead of time. (file: r5/deploy-checklist/SKILL.md)
- engineering:documentation: Write and maintain technical documentation. Trigger with "write docs for", "document this", "create a README", "write a runbook", "onboarding guide", or when the user needs help with any form of technical writing ? API docs, architecture docs, or operational runbooks. (file: r5/documentation/SKILL.md)
- engineering:incident-response: Run an incident response workflow ? triage, communicate, and write postmortem. Trigger with "we have an incident", "production is down", an alert that needs severity assessment, a status update mid-incident, or when writing a blameless postmortem after resolution. (file: r5/incident-response/SKILL.md)
- engineering:standup: Generate a standup update from recent activity. Use when preparing for daily standup, summarizing yesterday's commits and PRs and ticket moves, formatting work into yesterday/today/blockers, or structuring a few rough notes into a shareable update. (file: r5/standup/SKILL.md)
- engineering:system-design: Design systems, services, and architectures. Trigger with "design a system for", "how should we architect", "system design for", "what's the right architecture for", or when the user needs help with API design, data modeling, or service boundaries. (file: r5/system-design/SKILL.md)
- engineering:tech-debt: Identify, categorize, and prioritize technical debt. Trigger with "tech debt", "technical debt audit", "what should we refactor", "code health", or when the user asks about code quality, refactoring priorities, or maintenance backlog. (file: r5/tech-debt/SKILL.md)
- engineering:testing-strategy: Design test strategies and test plans. Trigger with "how should we test", "test strategy for", "write tests for", "test plan", "what tests do we need", or when the user needs help with testing approaches, coverage, or test architecture. (file: r5/testing-strategy/SKILL.md)
- github:gh-address-comments: Address actionable GitHub pull request review feedback. Use when the user wants to inspect unresolved review threads, requested changes, or inline review comments on a PR, then implement selected fixes. Use the GitHub app for PR metadata and flat comment reads, and use the bundled GraphQL script via `gh` whe (file: r8/gh-address-comments/SKILL.md)
- github:gh-fix-ci: Use when a user asks to debug or fix failing GitHub PR checks that run in GitHub Actions. Use the GitHub app from this plugin for PR metadata and patch context, and use `gh` for Actions check and log inspection before implementing any approved fix. (file: r8/gh-fix-ci/SKILL.md)
- github:github: Triage and orient GitHub repository, pull request, and issue work through the connected GitHub app. Use when the user asks for general GitHub help, wants PR or issue summaries, or needs repository context before choosing a more specific GitHub workflow. (file: r8/github/SKILL.md)
- github:yeet: Publish local changes to GitHub by confirming scope, committing intentionally, pushing the branch, and opening a draft PR through the GitHub app from this plugin, with `gh` used only as a fallback where connector coverage is insufficient. (file: r8/yeet/SKILL.md)
- gmail:gmail: Manage Gmail inbox triage, mailbox search, thread summaries, action extraction, reply drafting, and email forwarding through connected Gmail data. Use when the user wants to inspect a mailbox or thread, search email with Gmail query syntax, summarize messages, extract decisions and follow-ups, prepare replies (file: r9/gmail/SKILL.md)

- gmail:gmail-inbox-triage: Triage a Gmail inbox into actionable buckets such as urgent, needs reply soon, waiting, and FYI using connected Gmail data. Use when the user asks to triage the inbox, rank what needs attention, find what still needs a reply, or separate important mail from noise. (file: r9/gmail-inbox-triage/SKILL.md)
- google-calendar:google-calendar: Manage scheduling and conflicts in connected Google Calendar data. Use when the user wants to inspect calendars, compare availability, review conflicts, find a meeting room, review event notes or attachments, add or adjust reminders, place temporary holds, or draft exact create, update, reschedule, or canc (file: r10/google-calendar/SKILL.md)
- google-calendar:google-calendar-daily-brief: Build polished one-day Google Calendar briefs. Use when the user asks for today, tomorrow, or a specific date summary with an agenda, conflict flags, free windows, remaining-meeting readouts, or a calendar brief, and the Google Calendar connector is available. (file: r10/google-calendar-daily-brief/SKILL.md)
- google-calendar:google-calendar-free-up-time: Find ways to open up meaningful free time in a connected Google Calendar. Use when the user wants to clear up their day, make room for focus time, create a longer uninterrupted block, or see the smallest set of calendar changes that would give time back. (file: r10/google-calendar-free-up-time/SKILL.md)
- google-calendar:google-calendar-group-scheduler: Find and rank good meeting times for multiple people using connected Google Calendar data. Use when the user wants to schedule a group meeting, compare candidate slots across several attendees, find the best compromise time, or add a room check after narrowing the attendee-compatible options. (file: r10/google-calendar-group-scheduler/SKILL.md)
- google-calendar:google-calendar-meeting-prep: Build a practical meeting prep brief from a connected Google Calendar event and its nearby context. Use when the user wants to prepare for an upcoming meeting, understand what to read beforehand, pull in linked notes or docs, or get a concise brief on what the meeting appears to require. (file: r10/google-calendar-meeting-prep/SKILL.md)
- google-drive:google-docs: Connector-first Google Docs creation and editing in local Codex plugin sessions, with direct native create and batchUpdate workflows for simple docs, DOCX-first import for polished deliverables, target-document checks, smart chip and building-block reconstruction, connector-readback verification, and referenc (file: r11/google-docs/SKILL.md)
- google-drive:google-drive: Use connected Google Drive as the single entrypoint for Drive, Docs, Sheets, and Slides work. Use when the user wants to find, fetch, organize, share, export, copy, or delete Drive files, or summarize and edit Google Docs, Google Sheets, and Google Slides through one unified Google Drive plugin. (file: r11/google-drive/SKILL.md)
- google-drive:google-drive-comments: Write, reply to, and resolve Google Drive comments on Docs, Sheets, Slides, and Drive files with evidence-backed location context. Use when the user asks to leave comments, review a file with comments, respond to comment threads, or resolve Drive comments. (file: r11/google-drive-comments/SKILL.md)
- google-drive:google-sheets: Analyze and edit connected Google Sheets with range precision. Use when the user wants to create Google Sheets, find a spreadsheet, inspect tabs or ranges, search rows, plan formulas, create or repair charts, clean or restructure tables, write concise summaries, or make explicit cell-range updates. (file: r11/google-sheets/SKILL.md)
- google-drive:google-slides: Google Slides work for finding, reading, summarizing, creating, importing, template following, visual cleanup, source-deck adaptation, structural repair, and content edits in native Slides decks. (file: r11/google-slides/SKILL.md)
- pdf:pdf: Read, create, inspect, render, and verify PDF files where visual layout matters. Use Poppler rendering plus Python tools such as reportlab, pdfplumber, and pypdf for generation and extraction. (file: r13/pdf/26.630.12135/skills/pdf/SKILL.md)
- presentations:Presentations: Create or edit PowerPoint or Google Slides decks (file: r13/presentations/26.630.12135/skills/presentations/SKILL.md)
- productivity:memory-management: Two-tier memory system that makes Claude a true workplace collaborator. Decodes shorthand, acronyms, nicknames, and internal language so Claude understands requests like a colleague would. CLAUDE.md for working memory, memory/ directory for the full knowledge base. (file: r6/memory-management/SKILL.md)
- productivity:start: Initialize the productivity system and open the dashboard. Use when setting up the plugin for the first time, bootstrapping working memory from your existing task list, or decoding the shorthand (nicknames, acronyms, project codenames) you use in your todos. (file: r6/start/SKILL.md)
- productivity:task-management: Simple task management using a shared TASKS.md file. Reference this when the user asks about their tasks, wants to add/complete tasks, or needs help tracking commitments. (file: r6/task-management/SKILL.md)
- productivity:update: Sync tasks and refresh memory from your current activity. Use when pulling new assignments from your project tracker into TASKS.md, triaging stale or overdue tasks, filling memory gaps for unknown people or projects, or running a comprehensive scan to catch todos buried in chat and email. (file: r6/update/SKILL.md)

- skill-creator: Create new skills, modify and improve existing skills. Use when users want to create a skill from scratch, edit, or optimize an existing skill. (file: r0/skill-creator/SKILL.md)
- slack:slack: Read Slack context, route to the right Slack workflow, and prepare or perform Slack writes that match the user's intent. (file: r12/slack/SKILL.md)
- slack:slack-channel-summarization: Summarize activity from one Slack channel and return a concise recap, post-ready update, or summary doc. (file: r12/slack-channel-summarization/SKILL.md)
- slack:slack-daily-digest: Create a daily Slack digest from selected channels or topics. Use when the user asks for a daily Slack recap or summary of today's Slack activity. (file: r12/slack-daily-digest/SKILL.md)
- slack:slack-notification-triage: Triage recent Slack activity into a priority queue or task list for the user. (file: r12/slack-notification-triage/SKILL.md)
- slack:slack-outgoing-message: Primary skill for composing, drafting, or refining any outbound Slack content. Use this whenever the task will require using `slack_send_message`, `slack_send_message_draft`, or `slack_create_canvas`. Use `slack` to read or analyze Slack context; use this skill to produce the final outgoing message. (file: r12/slack-outgoing-message/SKILL.md)
- slack:slack-reply-drafting: Draft Slack replies from available context. Use when the user wants help finding messages that likely need a response and preparing reply drafts. (file: r12/slack-reply-drafting/SKILL.md)
- spreadsheets:Spreadsheets: Use this skill when a user requests to create, modify, analyze, visualize, or work with spreadsheet files (`.xlsx`, `.xls`, `.csv`, `.tsv`) or Google Sheets-targeted spreadsheet artifacts with formulas, formatting, charts, tables, and recalculation. (file: r13/spreadsheets/26.630.12135/skills/spreadsheets/SKILL.md)
- template-creator:template-creator: Create or update a reusable personal Codex artifact-template skill. Use when the user invokes $template-creator or asks in natural language to create a template using, from, or based on an attached Word document, PowerPoint presentation, or Excel workbook, or explicitly asks to edit or update a passed arti (file: r13/template-creator/26.630.12135/skills/template-creator/SKILL.md)

How to use skills

- Discovery: The list above is the skills available in this session (name + description + short path). Skill bodies live on disk at the listed paths after expanding the matching alias from `### Skill roots`.
- Trigger rules: If the user names a skill (with `$SkillName` or plain text) OR the task clearly matches a skill's description shown above, you must use that skill for that turn. Multiple mentions mean use them all. Do not carry skills across turns unless re-mentioned.
- Missing/blocked: If a named skill isn't in the list or the path can't be read, say so briefly and continue with the best fallback.
- How to use a skill (progressive disclosure):
  1) After deciding to use a skill, the main agent must expand the listed short `path` with the matching alias from `### Skill roots`, then open and read its `SKILL.md` completely before taking task actions. If a read is truncated or paginated, continue until EOF.
  2) When `SKILL.md` references relative paths (e.g., `scripts/foo.py`), resolve them relative to the directory containing that expanded `SKILL.md` first, and only consider other paths if needed.
  3) If `SKILL.md` points to extra folders such as `references/`, use its routing instructions to identify the files required for the task. The main agent must read each required instruction or reference file itself before acting on it. Do not delegate reading, summarizing, or interpreting skill instructions to a subagent. Subagents may still perform task work when the selected skill allows it.
  4) If `scripts/` exist, prefer running or patching them instead of retyping large code blocks.
  5) If `assets/` or templates exist, reuse them instead of recreating from scratch.
- Coordination and sequencing:
  - If multiple skills apply, choose the minimal set that covers the request and state the order you'll use them.
  - Announce which skill(s) you're using and why (one short line). If you skip an obvious skill, say why.
- Context hygiene:
  - Progressive disclosure applies to selecting relevant files, not partially reading a selected instruction file. Do not load unrelated references, scripts, or assets.
  - Avoid deep reference-chasing: prefer opening only files directly linked from `SKILL.md` unless you're blocked.
  - When variants exist (frameworks, providers, domains), pick only the relevant reference file(s) and note that choice.

- Safety and fallback: If a skill can't be applied cleanly (missing files, unclear instructions), state the issue, pick the next-best approach, and continue.
</skills_instructions>
<plugins_instructions>
Plugins
A plugin is a local bundle of skills, MCP servers, and apps.
How to use plugins
- Skill naming: If a plugin contributes skills, those skill entries are prefixed with `plugin_name:` in the Skills list.
- MCP naming: Plugin-provided MCP tools keep standard MCP identifiers such as `mcp__server__tool`; use tool provenance to tell which plugin they come from.
- Trigger rules: If the user explicitly names a plugin, prefer capabilities associated with that plugin for that turn.
- Relationship to capabilities: Plugins are not invoked directly. Use their underlying skills, MCP tools, and app tools to help solve the task.
- Relevance: Determine what a plugin can help with from explicit user mention or from the plugin-associated skills, MCP tools, and apps exposed elsewhere in this turn.
- Missing/blocked: If the user requests a plugin that does not have relevant callable capabilities for the task, say so briefly and continue with the best fallback.
</plugins_instructions>

2. user (2026-07-07T11:12:57.603Z)

AGENTS.md instructions

<INSTRUCTIONS>
Global instructions (all projects)

Repo-freshness convention: git pull BEFORE reading
Always `git pull` a repo before starting to read it ? at session start for the primary working directory, and again for any sibling/local repo the moment work touches it.

- **Why:** the owner closes every session on a "clean slate", but multiple parallel slates (sessions/machines) exist ? PRs get merged and master moves between sessions, so the local checkout can silently lag the remote truth. Pull first so ramp-up reads (handoff, docs, code) reflect where the remote actually is.
- **How (safe form):** `git pull --ff-only` on the current branch (a plain fetch+ff). Never auto-merge, rebase, or discard local state to make a pull succeed.
- **If it can't fast-forward** (diverged branch, dirty tree in the way): do NOT force anything ? report the state to the owner and proceed read-only on what's local, flagging that it may be stale. Uncommitted changes are often a sibling session's or the owner's WIP; never clobber them.

Git branching safety: concurrent sessions / shared checkouts
Multiple sessions ? and spawned/background tasks ? may share ONE working checkout and create branches + commit **concurrently**, so `git branch` / `git log` can change UNDER you between two commands. (Spawned "fresh worktree" tasks are NOT always isolated.) This has caused a real incident: a sibling commit landed in the gap between a branch-point check and `git checkout -b`, so the new branch rode on top of it and a **squash-merge silently absorbed the sibling's work**. Protocol ? do this EVERY time, not only when you suspect a collision:
- **Branch from the REMOTE, explicitly:** `git fetch origin` then `git checkout -b <name> origin/<base>`
  (e.g. `origin/master`). Never assume the current branch is the intended base.
- **Re-verify right before `git add`/commit:** `git branch --show-current` + `git log --oneline -1`
  (HEAD is yours). Before opening a PR, `git log --oneline origin/<base>..HEAD` must list ONLY your
  commits ? if a sibling commit appears, re-cut the branch from `origin/<base>`.
- **Stage explicit paths** (`git add <files>`) ? never `git add -A`/`.`/`-u`, so sibling or untracked
  files can't ride into your commit.
- **Serialize repo writes:** don't start a repo-writing spawned/background task while you hold

  uncommitted work ? commit or push first. If one may be running, treat the tree + current branch
  as able to shift, and put this branching-safety reminder into the spawned task's own prompt.

Session-handoff convention

Maintain a living **session handoff** for each project and use it to resume context quickly across windows.

- **Where it lives:**
  - If the project has an auto-memory dir (`~/.Codex/projects/<project>/memory/`), keep the handoff as `session-handoff.md` there, and list it under a **"? Start here"** entry at the top of that project's `MEMORY.md`.
  - Otherwise, keep a `SESSION_HANDOFF.md` at the repo root.
- **What it contains** (keep it to ~1 page ? it POINTS to detailed artifacts, never duplicates them):
  1. **You are here** ? current state.
  2. **Next steps / open threads.**
  3. **Ramp-up kit** ? the exact files/docs to read to get current.
  4. **Key decisions** ? so they aren't relitigated.
- **At session start:** if a handoff exists, read it before starting substantive work, and use it
  (plus its ramp-up kit) to get up to speed instead of replaying past conversation.
- **Updating it:** keep it current **automatically** ? refresh at meaningful checkpoints (a PR is
  pushed/merged, a key decision is made, work shifts phase, or a session is wrapping up), so a restart can ramp up without losing context. Do **not** rewrite it every turn ? only when something worth resuming from changed. The phrases **"update the handoff"** / **"refresh the handoff"** force an immediate full rewrite on demand.
- **Keep it honest:** it reflects real, shipped state ? never aspirational. (See each project's attribution/working-agreement note where present.)

Code review ? pre-LGTM gate (applies to every PR/code review, every project)

Do **not** post **LGTM** or otherwise sign off on a pull request until I have personally **verified** the following ? never trust the PR description's claims, reproduce them locally:

1. **Full test suite is green.** Run the whole suite (e.g. `run_all_tests.py` / `pytest`), not just the changed files.
2. **Install any dependency needed** so the suite actually collects and runs ? do not let modules skip or error for missing deps. (If a check genuinely can't run locally ? GPU/WSL/real-video/etc. ? say so explicitly and note what was and wasn't verified.)
3. **Lint + type checks are clean** ? `ruff check .` and `mypy ...`, matching the repo's CI. Install `ruff`/`mypy` if absent.
4. **Coverage stays up** ? run with coverage and confirm it stays **at or above the repo's floor** (e.g. the CI `--cov-fail-under` value) and does **not regress** versus the base branch.
5. **Review findings are resolved** ? bugs and repo-convention/discipline issues addressed.

LGTM is a *verified* stamp, never a rubber stamp. State plainly what was and wasn't verifiable locally.
</INSTRUCTIONS>
<environment_context>
  <cwd>C:\Users\avidu\Projects\Agent Sessions</cwd>
  <shell>powershell</shell>
  <current_date>2026-07-07</current_date>
  <timezone>Asia/Calcutta</timezone>
  <filesystem><workspace_roots><root>C:\Users\avidu\Projects\Agent Sessions</root></workspace_roots><permission_profile type="disabled"><file_system type="unrestricted" /></permission_profile></filesystem>
</environment_context>

3. user (2026-07-07T11:12:57.647Z)

Hello. Can you ramp up on the repo in the directory and starting with [https://github.com/avidullu/agent-sessions/issues/42](https://github.com/avidullu/agent-

sessions/issues/42) and make a project tracker for the same for execution? Additionally I want you to scan all the filed issues https://github.com/avidullu/agent-sessions/issues and identify any critical ones which can be folded into the project tracker that you are planning. Once the project is scoped then we can move to execution. I am happy to provide any feedbacks necessary

4. assistant (2026-07-07T11:13:05.365Z)

I?ll ramp up from the live repo state first, then read the GitHub workflow instructions and pull the issue context into a scoped tracker. I?ll keep this as planning/scope work for now, as requested, and won?t move into implementation.

5. assistant (2026-07-07T11:13:20.938Z)

The fast-forward pull found newer `origin/main`, but local `SESSION_HANDOFF.md` has uncommitted changes that would be overwritten. I?ll proceed on the local checkout and flag anything that may be stale, while using GitHub issue data directly for the current issue scan.

6. assistant (2026-07-07T11:13:26.344Z)

I?m using the `github:github` skill here because the core work is issue triage and repository orientation. The local checkout is 35 commits behind `origin/main` and blocked from fast-forward by a modified `SESSION_HANDOFF.md`, so I?ll treat local repo reads as potentially stale and lean on GitHub for the issue truth.

7. assistant (2026-07-07T11:13:31.391Z)

I have the GitHub connector available now. Next I?m going to pull issue #42 and the open issue set, then map the issue themes against the repo docs and current handoff.

8. assistant (2026-07-07T11:13:37.033Z)

The connector doesn?t expose a direct ?list all issues? shape in the first pass, so I?m checking whether the local `gh` client can give us the full issue inventory cleanly. That should let me preserve labels, bodies, comments, and state without scraping.

9. assistant (2026-07-07T11:13:53.014Z)

`gh` is authenticated for `avidullu`, so I can query the private repo directly. I?m going to grab issue #42 in full, plus the open issue inventory, then I?ll inspect the relevant docs/files named by the handoff.

10. assistant (2026-07-07T11:14:16.789Z)

The open issue set is compact: #18, #19, #23, #25, #26, #32, and #42. I?m reading the remote `origin/main` versions of the docs now, so the tracker is based on the branch GitHub currently sees rather than this stale working tree.

11. assistant (2026-07-07T11:14:24.797Z)

One notable mismatch already: issue #32 references `docs/TECH_DEBT_PLAN.md`, but that file is not present on `origin/main`. I?ll account for that as a tracker risk rather than assuming the issue text is fully current.

12. assistant (2026-07-07T11:14:37.443Z)

The README already has an agent-assisted setup prompt and final report shape, so #42 is not starting from zero. The gaps now look more precise: expected outputs per command, optional PDF/reportlab clarity, skipped inventory wording, and a better post-export/baseline next-step summary.

13. assistant (2026-07-07T11:14:45.716Z)

Issue #32 is important, but it?s explicitly the carry-forward from the completed tech-debt plan, not part of the first-user journey itself. I?m checking the CLI shape now to decide whether #42 should stay documentation-only or include a small command/output improvement slice.

14. assistant (2026-07-07T11:14:55.578Z)

I?m also going to apply the repo?s own tracked-project-doc convention here: issue #42 wants a scope/tracker before execution, and `docs/PROJECT_DOC_TEMPLATE.md` is the standard shape already used by this project.

15. assistant (2026-07-07T11:15:09.064Z)

Before I classify priorities, I?m checking whether any open issues have follow-up comments that change the scope. The bodies are detailed, but comments can hide owner decisions, so I don?t want to miss those.

16. assistant (2026-07-07T11:15:40.118Z)

I have enough to scope this. One more quick check: I?m capturing the exact local/remote commit state so the tracker notes are honest about what I inspected and why I?m not writing a repo doc yet.

17. assistant (2026-07-07T11:16:14.575Z)

I ramped up from GitHub issue state and `origin/main`. Local note: `git pull --ff-only` is blocked because `SESSION_HANDOFF.md` has local uncommitted changes; local `HEAD` is `5d81234`, while `origin/main` is `013170b`. I did not write files yet.

Proposed Project
`First-User Setup & Sync UX`
Primary issue: [#42](https://github.com/avidullu/agent-sessions/issues/42)
Tracker doc path once approved: `docs/FIRST_USER_SETUP_TRACKER.md`

Issue Scan
Fold in now:
- [#18](https://github.com/avidullu/agent-sessions/issues/18) ? direct overlap. New-computer setup and sync validation should be part of this project.

Critical but keep mostly separate:
- [#32](https://github.com/avidullu/agent-sessions/issues/32) ? important data-integrity work, but too broad/destructive for first-user setup. Fold in only as documentation/risk wording: do not imply `regenerate` exists yet; mention that archive backfill is tracked separately.

Do not fold into this project:
- [#19](https://github.com/avidullu/agent-sessions/issues/19) provenance traces ? useful for baseline trust, not setup UX.
- [#23](https://github.com/avidullu/agent-sessions/issues/23), [#25](https://github.com/avidullu/agent-sessions/issues/25), [#26](https://github.com/avidullu/agent-sessions/issues/26) ? larger learning/baseline features.

Draft Tracker
ID	Deliverable	Source	Status
P0	Add tracked project doc and index/handoff pointers	repo convention	Todo
P1	README ?expected outputs? for `discover`, `status`, `export`, `baseline scaffold`, `baseline suggest`	#42	Todo
P2	Clarify Markdown vs PDF behavior, `reportlab`, and how to verify PDF support	#42	Todo
P3	Explain inventory-only skipped sources as expected behavior, especially Copilot/ZAI	#42	Todo
P4	Add concise post-export / post-baseline next-step guidance	#42	Todo
P5	Add `docs/NEW_MACHINE_SETUP.md` or expand `docs/MULTI_MACHINE.md` with copy/paste setup + validation checklist	#18	Todo
P6	Add troubleshooting for large `git status`, dirty/diverged git state, missing WSL paths,

missing PDF support, local `sources.toml` | #18/#42 | Todo |
| P7 | Optional stretch: read-only `doctor` / `setup` command for Python/dev deps/reportlab/source roots/extractor availability/git freshness | #42/#18 | Stretch |
| P8 | Document boundary to #32: archive backfill/regenerate remains separate and must be backup/dry-run driven | #32 | Todo |

Execution Slices
1. Docs MVP: tracker doc, README expected outputs, PDF/inventory wording.
2. New-machine guide: checklist + troubleshooting from #18.
3. CLI polish: improve final export summary and tests, only if docs alone feel insufficient.
4. Stretch: `doctor` command.

My recommendation: approve P0-P6 + P8 as the scoped MVP, keep P7 as stretch, and leave #32 as its own project except for boundary notes.

18. user (2026-07-07T11:23:42.441Z)

LGTM to your recommendation. Please document this decision, update the docs and trackers and only after that start sending the PRs and doing a self review audit and merging them with improved test coverage and ruff/mypy resolutions

19. assistant (2026-07-07T11:23:56.614Z)

Great, I?ll turn that recommendation into the repo?s tracked-project docs first. Because the main checkout is behind and has a dirty `SESSION_HANDOFF.md`, I?m going to work from a fresh branch/worktree rooted at `origin/main` so I don?t disturb that local WIP.

20. assistant (2026-07-07T11:24:04.621Z)

Fetch is clean. Next I?m creating an isolated worktree branch from `origin/main`; that gives us the latest repo truth while leaving the existing dirty handoff untouched.

21. assistant (2026-07-07T11:24:09.450Z)

Branch name and target path are available, so I?m cutting the work branch now from `origin/main` exactly.

22. assistant (2026-07-07T11:24:32.716Z)

The isolated branch is ready at `C:\Users\avidu\Projects\Agent Sessions - first-user-setup-ux`. I?m doing the repo-freshness pull there too before editing, then I?ll create the tracker and documentation updates.

23. assistant (2026-07-07T11:24:49.507Z)

I?m splitting this into two small PRs: first the decision and first-user/new-machine docs, then a CLI summary/test-coverage polish slice. I?m reading the target docs in the fresh worktree before patching so the edits fit the current wording.

24. assistant (2026-07-07T11:25:03.534Z)

I?m going to make the docs PR carry the approved decision plus the user-facing setup guidance. The tracker will explicitly mark #18 as folded in, #32 as a boundary/risk, and the optional `doctor` command as a later code slice.

25. assistant (2026-07-07T11:25:43.599Z)

The tracked project doc is in place. I?m adding the practical new-machine checklist next, with explicit validation and troubleshooting so #18 is covered inside the approved setup project.

26. assistant (2026-07-07T11:26:16.334Z)

Now I?m updating the README quick path so a first user sees expected changed files, PDF

behavior, inventory skips, and the immediate next steps after export/baseline suggestion.

27. assistant (2026-07-07T11:26:36.475Z)

README now names what should change and why skipped inventory sources are okay. I?m wiring the new tracker/checklist into the docs index, multi-machine doc, and session handoff so future sessions start in the right place.

28. assistant (2026-07-07T11:26:59.126Z)

The docs are wired in. I?m doing a quick local review of the diff now, then I?ll run the repo gates even though this first slice is documentation-only.

29. assistant (2026-07-07T11:27:11.680Z)

The diff looks scoped and the main README now has the missing ?what should happen?? layer. I?m running the full local gates before committing this docs slice, because the repo?s review discipline asks for verified green even on docs work.

30. assistant (2026-07-07T11:27:37.615Z)

Docs slice gates are green: 404 tests passed with 96.54% coverage, ruff clean, mypy clean. I?m doing the branch/HEAD recheck now, then I?ll stage only the intended docs files and commit.

31. assistant (2026-07-07T11:27:41.188Z)

HEAD is exactly `origin/main` plus no local commits yet, and whitespace checks are clean. I?m staging explicit documentation paths only.

32. assistant (2026-07-07T11:27:47.744Z)

The docs files are staged. I?m committing this first slice as the tracker/setup documentation PR payload.

33. assistant (2026-07-07T11:27:50.815Z)

Staging contains only the six intended docs/handoff files. Committing now.

34. assistant (2026-07-07T11:27:57.191Z)

Docs slice is committed. I?m pushing the branch and opening a PR now; after the PR exists I?ll self-review the exact diff before merging.

35. assistant (2026-07-07T11:28:08.062Z)

Branch is pushed. I?m opening the docs PR with the verified gate results in the description.

36. assistant (2026-07-07T11:28:24.226Z)

PR #43 is open. I?m updating the tracker inside that PR with the actual PR number and marking the docs deliverables as the PR?s completion target, so the tracker won?t be vague after merge.

37. assistant (2026-07-07T11:28:46.724Z)

Small honesty pass: I?m keeping the tracker status as ?In review? until #43 actually lands, rather than pretending the docs slice is already merged. After merge, the next code slice can mark it done.

38. assistant (2026-07-07T11:29:04.373Z)

Tracker status is now honest for an open PR. I?m amending the docs commit with that PR-number update.

39. assistant (2026-07-07T11:29:13.162Z)

The commit is amended. I?m re-running the full local gates on the exact PR head before the self-review/merge step.

40. assistant (2026-07-07T11:29:31.656Z)

The exact PR head is green again: 404 passed, coverage 96.54%, ruff clean, mypy clean. I?m force-with-lease pushing the amended docs commit, then I?ll self-review the PR diff and check CI before merge.

41. assistant (2026-07-07T11:29:43.475Z)

PR #43 is updated. I?m doing a self-review pass now: changed files, branch contents versus `origin/main`, and CI state.

42. assistant (2026-07-07T11:29:50.963Z)

Self-review scope looks right: one commit, six intended files, no archive/raw/config drift. GitHub CI is still pending, so I?m waiting for that check to finish before merging.

43. assistant (2026-07-07T11:30:23.270Z)

CI passed too. I?m recording the self-review summary on the PR, then I?ll merge it with a normal merge commit to match the repo?s recent PR history.

44. assistant (2026-07-07T11:30:26.684Z)

Branch verification is clean: PR #43 contains only my docs commit over `origin/main`. I?m posting the self-review note and merging now.

45. assistant (2026-07-07T11:30:35.917Z)

Self-review note is on the PR. Merging #43 now.

46. assistant (2026-07-07T11:30:46.272Z)

PR #43 is merged and its branch was deleted remotely. I?m moving to the code/test polish slice now from the newly updated `origin/main`, again cutting the branch directly from the remote base.

47. assistant (2026-07-07T11:31:01.732Z)

`origin/main` now includes PR #43. I?m creating the code-slice branch from that remote head, not from the previous docs branch.

48. assistant (2026-07-07T11:31:24.666Z)

For the code slice, I?m keeping the blast radius small: make `export` end with an actionable setup summary, and add CLI tests around the skipped-source/PDF/dry-run branches that were previously thinly covered. That directly addresses #42 without pulling the optional `doctor` command forward prematurely.

49. assistant (2026-07-07T11:31:51.215Z)

I?m adding the export summary as a small helper so it?s easy to test without fragile whole-command output assertions, then adding a main-level test for the PDF-missing/skipped-source branch.

50. assistant (2026-07-07T11:32:24.965Z)

The CLI now prints an actionable export summary, and the tests cover dry-run, skipped inventory, PDF-missing, raw/PDF next-step branches. I?m running the targeted CLI tests first so any output

expectations shake out quickly.

51. assistant (2026-07-07T11:32:38.736Z)

Targeted CLI tests pass, and CLI coverage for that module is up to 98.03%. Since #43 is now merged, I?m updating the tracker to mark the docs slice done and add this code slice as the next tracked row.

52. assistant (2026-07-07T11:33:05.419Z)

The tracker now reflects #43 as merged and P9 as the active code slice. I?m running the full test/ruff/mypy gates now before committing.

53. assistant (2026-07-07T11:33:29.292Z)

Full gates are green on the code slice: 408 tests passed, total coverage 96.72%, ruff clean, mypy clean. I?m doing the branch/HEAD recheck, then staging only the code, tests, tracker, and handoff files for this commit.

54. assistant (2026-07-07T11:33:33.781Z)

HEAD is the merged docs PR with no local commits yet, and the diff is clean. Staging explicit paths now.

55. assistant (2026-07-07T11:33:40.081Z)

Only the intended code/test/tracker files are staged. I?m committing the code slice now.

56. assistant (2026-07-07T11:33:43.396Z)

Staging is exactly the four expected files. Committing.

57. assistant (2026-07-07T11:33:49.653Z)

The code slice is committed. I?m pushing it and opening the next PR with the coverage delta and gates in the description.

58. assistant (2026-07-07T11:33:55.612Z)

Branch is pushed. Opening PR now.

59. assistant (2026-07-07T11:34:08.475Z)

PR #44 is open. I?m updating the tracker row from ?code slice? to the actual PR number before final self-review.

60. assistant (2026-07-07T11:34:16.194Z)

Tracker now points at #44 honestly as ?In review.? I?m amending that into the PR commit, then I?ll re-run the full gates on the exact amended head.

61. assistant (2026-07-07T11:34:23.864Z)

Amended. Re-running full verification now, then I?ll push the amended branch.

62. assistant (2026-07-07T11:34:41.657Z)

Exact amended PR head is green: 408 passed, coverage 96.72%, ruff clean, mypy clean. I?m pushing the amended commit and then waiting for CI/self-review before merge.

63. assistant (2026-07-07T11:34:52.512Z)

PR #44 is updated. I?m checking its commit/file set and CI state now.

64. assistant (2026-07-07T11:34:57.892Z)

The PR has the intended one commit and four files. CI is pending, so while it runs I?m doing a self-review of the code diff itself.

65. assistant (2026-07-07T11:35:17.109Z)

One tracker nuance: once #44 merges, the MVP is complete and the optional `doctor` command stays deferred. I?m going to make that final state explicit in the PR before merging, then re-run/push once more so the merged tracker is not stale.

66. assistant (2026-07-07T11:35:31.316Z)

I caught one wording mismatch in the handoff: if the MVP is done after #44, it should be listed as completed rather than active. Fixing that before the final amend.

67. assistant (2026-07-07T11:35:36.877Z)

Final tracker/handoff wording is set. I?m amending the PR commit again and running the exact-head gates one last time.

68. assistant (2026-07-07T11:35:44.576Z)

Amended. Running final full verification now.

69. assistant (2026-07-07T11:36:01.616Z)

Final local gates are green again. I?m force-with-lease pushing the final amended PR head now.

70. assistant (2026-07-07T11:36:10.846Z)

The final PR head is pushed. I?m waiting for CI, then I?ll post the self-review note and merge if everything stays green.

71. assistant (2026-07-07T11:36:53.149Z)

CI passed. I?m doing the final branch/commit verification, then posting the self-review audit and merging #44.

72. assistant (2026-07-07T11:36:59.040Z)

Final verification is clean: #44 has one intended commit over `origin/main`, merge state is clean, and CI is green. I?m posting the self-review note and merging.

73. assistant (2026-07-07T11:37:08.157Z)

Self-review note is posted. Merging #44 now.

74. assistant (2026-07-07T11:37:31.098Z)

Both PRs are merged. I?m refreshing `origin/main` and checking the final remote state so the wrap-up is accurate.

75. assistant (2026-07-07T11:37:45.719Z)

PRs #43 and #44 are both merged on `origin/main`. I?m closing the two completed issues now with a short completion note; #32 stays open because it was deliberately kept separate.

76. assistant (2026-07-07T11:38:17.689Z)

Done. I documented the decision, updated the trackers/docs, executed the scoped work in two PRs, self-reviewed both, waited for CI, and merged them.

Merged:
- [PR #43](https://github.com/avidullu/agent-sessions/pull/43): setup UX tracker, new-machine setup guide, README expected outputs, PDF/inventory guidance, handoff/docs index updates.
- [PR #44](https://github.com/avidullu/agent-sessions/pull/44): CLI export next-steps summary plus expanded CLI tests.

Verification:
- Docs PR: 404 tests passed, coverage 96.54%, ruff clean, mypy clean, CI passed.
- Code PR: 408 tests passed, coverage 96.72%, CLI targeted coverage 98.03%, ruff clean, mypy clean, CI passed.

I also closed issues #42 and #18 with completion notes. #32 remains open as the separate archive backfill/regenerate project, per the decision.

One workspace note: the original checkout at `C:\Users\avidu\Projects\Agent Sessions` still has the pre-existing dirty `SESSION_HANDOFF.md`; all repo writes were done safely in the isolated worktree `C:\Users\avidu\Projects\Agent Sessions - first-user-setup-ux`.

::git-create-branch{cwd="C:/Users/avidu/Projects/Agent Sessions - first-user-setup-ux" branch="codex/first-user-setup-ux"}
::git-create-branch{cwd="C:/Users/avidu/Projects/Agent Sessions - first-user-setup-ux" branch="codex/setup-summary-polish"}
::git-stage{cwd="C:/Users/avidu/Projects/Agent Sessions - first-user-setup-ux"}
::git-commit{cwd="C:/Users/avidu/Projects/Agent Sessions - first-user-setup-ux"}
::git-push{cwd="C:/Users/avidu/Projects/Agent Sessions - first-user-setup-ux" branch="codex/first-user-setup-ux"}
::git-push{cwd="C:/Users/avidu/Projects/Agent Sessions - first-user-setup-ux" branch="codex/setup-summary-polish"}
::git-create-pr{cwd="C:/Users/avidu/Projects/Agent Sessions - first-user-setup-ux" branch="codex/first-user-setup-ux" url="https://github.com/avidullu/agent-sessions/pull/43" isDraft=false}
::git-create-pr{cwd="C:/Users/avidu/Projects/Agent Sessions - first-user-setup-ux" branch="codex/setup-summary-polish" url="https://github.com/avidullu/agent-sessions/pull/44" isDraft=false}

77. user (2026-07-07T11:40:31.441Z)

Thanks. Can you scope out and make design a plan based on some research on how to approach the #23, #25 and #26.

78. assistant (2026-07-07T11:40:45.528Z)

I?ll treat this as a planning/design pass, not feature implementation. First I?ll refresh repo context safely, then pull the current issue text, do a research pass on the relevant patterns, and turn it into a tracked project/design plan in the repo.

79. assistant (2026-07-07T11:41:01.759Z)

The primary checkout is still blocked from fast-forward by the existing dirty `SESSION_HANDOFF.md`, so I won?t touch it. I?ll use a fresh worktree from `origin/main` for this planning doc, keeping the local WIP isolated. I?m using the GitHub, system-design, and documentation skills because this is issue triage plus a technical design document.

80. assistant (2026-07-07T11:41:18.747Z)

I?m going to make this a planning-doc branch rather than changing feature code. Next I?m creating an isolated worktree from `origin/main`, then I?ll pull issue details and do the external research pass.

81. assistant (2026-07-07T11:41:22.366Z)

The branch/path are clear. I?m cutting the design-plan worktree from the current remote main.

82. assistant (2026-07-07T11:41:42.113Z)

The design branch is ready at `C:\Users\avidu\Projects\Agent Sessions - knowledge-replay-design-plan`. I?m pulling issue bodies/comments and the relevant repo docs now.

83. assistant (2026-07-07T11:41:54.774Z)

The three issues are tightly coupled: #26 gives the durable schema/wiki layer, #23 is one evidence source that should feed it, and #25 is the more ambitious replay signal generator. I?m reading the existing baseline/composition docs next so the plan lands in the current architecture instead of inventing a parallel system.

84. assistant (2026-07-07T11:44:01.987Z)

The research is pointing toward a clear sequencing: build the schema/compiled knowledge substrate first (#26), feed the safest deterministic signal source into it next (#23 handoffs), and only then add replay selection/bundling (#25). That keeps the ambitious loop grounded instead of making replay the first moving part.

85. assistant (2026-07-07T11:45:41.918Z)

I?ve got the shape of the plan from the issue read-through and the research pass. I?m going to turn that into a tracked design doc now, update the docs index and handoff, then run a light docs-only verification so there?s something concrete for you to react to.

86. assistant (2026-07-07T11:45:51.891Z)

The isolated planning worktree is still cleanly up to date with `origin/main`, so I?m staying out of the dirty primary checkout. I?m checking the existing docs index and handoff shape before patching so the new tracker lands in the repo?s current style.

87. assistant (2026-07-07T11:46:00.595Z)

The current docs index treats active design trackers as first-class docs, and the handoff points people through a short ramp-up kit. I?m adding this as the same kind of living tracker, with explicit sequencing from ?schema/wiki layer? to ?handoff mining? to ?replay ingestion.?

88. assistant (2026-07-07T11:46:25.411Z)

I?m writing the plan as a design-and-execution tracker rather than a one-off proposal. The important sequencing decision I?m documenting is: build the schema/compiled wiki layer first, use handoff mining as the first producer, and only then add replay because replay needs provenance, redaction, and bundle contracts to be boring and reviewable.

89. assistant (2026-07-07T11:48:07.249Z)

The plan doc, docs index, and handoff pointer are patched in the isolated branch. I?m doing a quick consistency pass now: checking the diff, whitespace, and whether the tracker reads like an executable sequence rather than a pile of ideas.

90. assistant (2026-07-07T11:48:31.604Z)

The first pass is clean on whitespace. I noticed K0 is now materially complete in the branch, so I?m going to tighten that status and then run the repo?s docs-safe checks before deciding whether this should stay local or become a review PR.

91. assistant (2026-07-07T11:48:42.603Z)

K0 now reflects the actual local state: the plan, index, and handoff pointer are in place. I?m checking the repo?s configured verification commands next so I don?t guess at the test/lint surface.

92. assistant (2026-07-07T11:48:50.038Z)

The repo advertises the full verification set in `README.md` and CI: pytest with coverage, ruff,

and mypy. Even though this is docs-only, I?m going to run the same gates so a future plan PR has a clean factual verification note.

93. assistant (2026-07-07T11:49:12.360Z)

Pytest is green: 408 passed with 96.72% coverage, above the repo?s 80% floor. I?m running ruff and mypy now, then I?ll decide whether to package this as a review PR or leave it as a local scoped branch for your feedback.

94. assistant (2026-07-07T11:49:23.563Z)

Ruff and mypy are clean too, and the whitespace check is still quiet. I?m checking for any tool-generated files before I either commit the docs branch or keep the worktree ready for review.

95. assistant (2026-07-07T11:49:40.864Z)

The test tools left only ignored local caches and `.coverage`; the actual changes are the new plan doc plus the docs index and handoff. I?m going to commit those three explicit paths on the planning branch so the scoped work is stable and reviewable.

96. assistant (2026-07-07T11:49:46.462Z)

Branch safety looks good: I?m on `codex/knowledge-replay-design-plan`, HEAD is the current `origin/main` merge commit, and there are no stray commits on the branch yet. Staging only the three docs paths now.

97. assistant (2026-07-07T11:49:51.358Z)

The staged set is exactly the plan doc, docs index, and handoff update. I?m committing that scoped documentation change now.

98. assistant (2026-07-07T11:49:56.015Z)

The commit is created. Since this is a design-review artifact and not implementation, I?m going to push it and open a PR for feedback rather than merge anything behind your back.

99. assistant (2026-07-07T11:50:04.466Z)

The branch has exactly one commit over `origin/main`, which is the plan doc commit. Pushing that branch now so the PR can be reviewed normally.

100. assistant (2026-07-07T11:50:13.890Z)

Branch is on GitHub. I?m opening this as a draft PR because it?s the scoping/design artifact for your feedback, not a code slice to merge immediately.

101. assistant (2026-07-07T11:50:35.620Z)

PR #45 is open as a draft. I?m doing a quick self-review pass against the PR diff and checking whether GitHub has started any CI checks, then I?ll hand you the review-ready summary.

102. assistant (2026-07-07T11:50:42.402Z)

The draft PR changes only the three intended docs files. GitHub CI has started and is currently in progress, so I?m going to wait for that result rather than leaving you with a half-known PR state.

103. assistant (2026-07-07T11:51:16.193Z)

CI passed. One small bookkeeping thing before I call this done: now that PR #45 exists, I?m updating the tracker and handoff to point at it so the branch remains self-describing when reviewed later.

104. assistant (2026-07-07T11:51:27.873Z)

The PR metadata is now reflected in the tracker and handoff. I?m rerunning the same local gates after that amend so the verification note remains exact.

105. assistant (2026-07-07T11:51:31.915Z)

Whitespace is clean after the PR-link update. Running the full repo checks one more time; they were already green, but this keeps the audit trail tidy.

106. assistant (2026-07-07T11:51:53.133Z)

The full suite is still green after the handoff/tracker bookkeeping: 408 passed, 96.72% coverage. Final lint/type pass is running, then I?ll amend and push the branch update.

107. assistant (2026-07-07T11:52:01.637Z)

Ruff and mypy stayed clean. I?m amending the existing docs commit with only the PR-link bookkeeping, then force-pushing with lease so PR #45 stays as one tidy design commit.

108. assistant (2026-07-07T11:52:04.877Z)

The branch still contains only the intended design commit over `origin/main`. Staging the two amended docs paths explicitly now.

109. assistant (2026-07-07T11:52:09.901Z)

The amended commit is in place. I?m force-pushing with lease, then I?ll wait for PR #45?s CI to settle again.

110. assistant (2026-07-07T11:52:19.729Z)

The updated branch is pushed. GitHub will usually start a fresh check after the force-push; I?m checking that status now and will wait if it?s running.

111. assistant (2026-07-07T11:52:24.853Z)

CI is queued on the amended PR. I?m waiting for it so the final status can be crisp: either passed or exactly what failed.

112. assistant (2026-07-07T11:53:30.178Z)

Scoped and documented the design plan in [docs/BASELINE_KNOWLEDGE_REPLAY_PLAN.md](<C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan/docs/BASELINE_KNOWLEDGE_REPLAY_PLAN.md>), updated [docs/README.md](<C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan/docs/README.md>), and refreshed [SESSION_HANDOFF.md](<C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan/SESSION_HANDOFF.md>).

Opened draft [PR #45](https://github.com/avidullu/agent-sessions/pull/45) for review. The recommended sequence is: `baseline/SCHEMA.md` + deterministic wiki/project-page layer first, handoff mining next, replay select/bundle/ingest after provenance and redaction contracts. I also folded #19 in as a provenance substrate and kept #32 out as a boundary issue.

Verification/self-review:
- Local: `git diff --check`, `pytest --cov` 408 passed with 96.72% coverage, `ruff`, `mypy`
- GitHub CI: passed
- PR diff only touches the three intended docs files

Research sources are cited in the plan, including [LLM Wiki](https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f), [Reflexion](https://arxiv.org/abs/2303.11366), [Generative Agents](https://arxiv.org/abs/2304.03442), [AgentRR](https://arxiv.org/abs/2505.17716),

[AgentHER](https://arxiv.org/abs/2603.21357), [OpenAI agent evals](https://developers.openai.com/api/docs/guides/agent-evals), and [W3C PROV-DM](https://www.w3.org/TR/prov-dm/).

::git-create-branch{cwd="C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan" branch="codex/knowledge-replay-design-plan"}
::git-stage{cwd="C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan"}
::git-commit{cwd="C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan"}
::git-push{cwd="C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan" branch="codex/knowledge-replay-design-plan"}
::git-create-pr{cwd="C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan" branch="codex/knowledge-replay-design-plan" url="https://github.com/avidullu/agent-sessions/pull/45" isDraft=true}

113. user (2026-07-07T12:02:17.746Z)

pls review the PR comments

114. assistant (2026-07-07T12:02:27.919Z)

I?ll review PR #45?s comments and treat this as an address-comments pass: first inspect the review threads and CI state, then make any scoped doc updates if the feedback is actionable.

115. assistant (2026-07-07T12:02:38.959Z)

I?m using the `github:gh-address-comments` workflow here, since PR comments can have thread state that flat comment views miss. First I?m refreshing the PR worktree safely, then I?ll pull thread-aware review data.

116. assistant (2026-07-07T12:02:45.484Z)

The PR worktree is up to date and clean, and `gh` is authenticated. I?m resolving the current branch?s PR number and reading both top-level and inline review-thread context next.

117. assistant (2026-07-07T12:03:06.452Z)

The GitHub connector can?t see this private repo, but the local `gh` CLI can and confirms PR #45 is clean with CI success. I?m falling back to `gh api graphql` for review threads so we still get resolved/outdated state.

118. assistant (2026-07-07T12:03:12.241Z)

There are no top-level issue comments, but there is a substantive owner review on PR #45. I?m checking whether any inline threads exist; then I?ll patch the design doc for the two must-address items and the cheap consistency fixes.

119. assistant (2026-07-07T12:03:27.298Z)

No inline review threads exist, so the actionable feedback is all in the single review body. I?m reading the referenced code and trackers now so the fix reconciles with what?s already shipped rather than just wording around it.

120. assistant (2026-07-07T12:03:52.837Z)

The review has two real blockers and several consistency nits. I?m making the doc point at existing ownership instead of creating parallel work: P6/P10/P11/P5 in the loop tracker, the already-shipped marker/upsert machinery, and the replay R1-R5 gate names from issue #25.

121. assistant (2026-07-07T12:04:31.659Z)

I?ve got the concrete edits lined up: add a tracker-reconciliation subsection, switch the example to the existing `baseline:begin/end` grammar, make K2 reuse the shipped upsert helper, restore issue #25?s R1-R5 gate names, and name how schema validation is actually enforced.

122. assistant (2026-07-07T12:05:24.619Z)

The main plan doc now reuses the existing marker/upsert path and maps K3/K8 back to the existing P rows. I?m adding small cross-links in the baseline loop tracker too, so the ?source of truth? documents point at each other instead of drifting.

123. assistant (2026-07-07T12:05:38.485Z)

The loop tracker now names the knowledge/replay plan as the consumer or implementation slice for P5/P6/P11. I?m cleaning up the tracker metadata and then I?ll run a local doc consistency pass.

124. assistant (2026-07-07T12:05:51.172Z)

The review-driven changes are now in the docs. I?m adding the changelog/reference breadcrumbs so a future session understands why the plan tightened around existing code and tracker ownership.

125. assistant (2026-07-07T12:06:00.187Z)

One ordering nit in the changelog jumped out while reviewing my own patch; I?m fixing that before running checks.

126. assistant (2026-07-07T12:06:21.503Z)

One dependency label was a little too narrow: the review named the P10/P11 design as the substrate, even though P11 carries most of the blocking redaction/dedup mechanics. I?m making the row explicit to avoid ambiguity.

127. assistant (2026-07-07T12:06:37.661Z)

The cleanup scan didn?t find the old forked marker grammar or manifest wording. I?m updating the handoff with the review-addressed state before running the full verification set.

128. assistant (2026-07-07T12:06:48.041Z)

The branch now records the review response in the handoff as well. I?m running the repo gates again: whitespace, full tests with coverage, ruff, and mypy.

129. assistant (2026-07-07T12:06:54.695Z)

Whitespace, ruff, and mypy are clean. The full pytest coverage run is the last local gate.

130. assistant (2026-07-07T12:07:22.450Z)

All local gates are green after the review fixes: 408 tests pass with 96.72% coverage, plus ruff/mypy clean. I?m committing the explicit docs/handoff paths now and pushing them to PR #45.

131. assistant (2026-07-07T12:07:29.699Z)

Branch safety is clean: still on `codex/knowledge-replay-design-plan`, with only the PR?s docs commit over `origin/main`. I?m staging just the three changed documentation paths.

132. assistant (2026-07-07T12:07:34.146Z)

The review-response commit is created. I?m pushing it normally to the existing PR branch and then I?ll watch CI.

133. assistant (2026-07-07T12:07:43.868Z)

The PR branch is updated. I?m checking the PR status and waiting for GitHub Actions to finish.

134. assistant (2026-07-07T12:07:48.069Z)

PR #45 is updated and CI is queued. I?ll wait for the check to finish before I summarize the

review response.

135. assistant (2026-07-07T12:08:38.851Z)

Reviewed PR #45 comments and pushed fixes in `edbb14b`.

The PR had one top-level design review and no inline review threads. I addressed the review by updating:

- [docs/BASELINE_KNOWLEDGE_REPLAY_PLAN.md](<C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan/docs/BASELINE_KNOWLEDGE_REPLAY_PLAN.md>): mapped K3/K8 back to P6/P10/P11/P5, reframed K2 to reuse `upsert_promoted_content()`, restored #25?s R1-R5 replay gates, clarified schema validation, fixed marker grammar, manifest path, project slug derivation, CLI convention, and automation stance.
- [docs/BASELINE_LOOP_CLOSURE.md](<C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan/docs/BASELINE_LOOP_CLOSURE.md>): cross-linked P5/P6/P11 to the knowledge/replay plan so ownership is not forked.
- [SESSION_HANDOFF.md](<C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan/SESSION_HANDOFF.md>): noted the review-addressed state.

Verification:
- `git diff --check`
- `python -m pytest --cov=agent_sessions --cov-report=term-missing` ? 408 passed, 96.72% coverage
- `python -m ruff check .`
- `python -m mypy agent_sessions tools`
- GitHub CI for PR #45 passed

I did not post GitHub replies or resolve anything because the review was a top-level comment, not an inline thread.

::git-stage{cwd="C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan"}
::git-commit{cwd="C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan"}
::git-push{cwd="C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan" branch="codex/knowledge-replay-design-plan"}

136. user (2026-07-07T12:15:02.857Z)

Did you post a response on the PR?

137. assistant (2026-07-07T12:15:10.698Z)

No. I only pushed the review-fix commit to PR #45; I did not post a PR comment or reply.

I held off because the review was a top-level comment, not an inline thread, and I don?t write GitHub replies unless you explicitly ask.

138. user (2026-07-07T12:15:40.699Z)

Please post a detailed comment so the reviewer can take another pass

139. assistant (2026-07-07T12:15:52.938Z)

I?ll post a PR comment now with a response matrix tied to the review points and the verification I ran. I?ll refresh the PR branch first so the comment references the current pushed state.

140. assistant (2026-07-07T12:16:02.389Z)

I?m using the PR-comment workflow, and this time you?ve explicitly asked for the GitHub write. I?m checking the branch and PR metadata, then I?ll post one top-level comment.

141. assistant (2026-07-07T12:16:09.416Z)

The PR is clean, current branch has the two expected commits, and CI is green. I?m posting a

detailed response matrix now so the reviewer can quickly re-check the changes.

142. assistant (2026-07-07T12:16:39.830Z)

Posted a detailed PR comment for the reviewer:
[issuecomment-4903687464](https://github.com/avidullu/agent-sessions/pull/45#issuecomment-4903687464).

It includes a response matrix for the must-address items, the smaller fixes, changed files, and verification results.

143. user (2026-07-07T12:23:21.895Z)

Please review the comments and respond accordingly

144. assistant (2026-07-07T12:23:34.892Z)

I?ll do another PR-comments pass: refresh the branch, read the latest top-level comments/reviews and any inline threads, then either patch the docs or post a response depending on what changed.

145. assistant (2026-07-07T12:23:44.022Z)

I?m using the GitHub PR comment handler again. The connector previously couldn?t see this private repo, so I?ll use `gh` for authoritative PR comments and GraphQL for any review-thread state.

146. assistant (2026-07-07T12:23:52.575Z)

There are two newer review submissions, plus a consolidated punch-list. Several items are already covered by the last commit, but the new P0 items add two important design changes: make redaction its own first-class deliverable and add ingest provenance-integrity validation. I?m checking thread state, then I?ll patch the plan.

147. assistant (2026-07-07T12:24:05.529Z)

There are still no inline threads, so the actionable source is the consolidated top-level review. I?m going to treat P0 and P1 as required doc changes, and fold the P2 clarifications in while I?m there.

148. assistant (2026-07-07T12:24:15.776Z)

The code confirms the reviewer?s ingest point: `validate_proposal()` is shallow, and `Prediction` currently drops unknown provenance fields. I?m pulling #19?s wording now so the lightweight provenance fields line up with the intended trace vocabulary instead of inventing a parallel envelope.

149. assistant (2026-07-07T12:24:57.405Z)

I?m going to make the safety and trust boundaries explicit in the tracker rather than leaving them as ?future implementation wisdom.? The concrete changes are: a first-class redaction slice, a provenance/trace contract aligned to #19, a separate proposal/Prediction provenance extension, ingest-side archive reference validation, and a read-only handoff audit slice that can land early.

150. assistant (2026-07-07T12:25:18.703Z)

The top-level thesis and architecture now show redaction as an explicit preflight and remove the misleading lint-to-promote edge. Next I?m tightening the data contracts: trace fields, handoff extraction behavior, and ingest integrity.

151. assistant (2026-07-07T12:25:49.413Z)

The handoff example no longer duplicates provenance fields, and replay ingest now has explicit

archive-reference validation requirements. I?m adding the redaction v0 design and the replay value/kill criteria now.

152. assistant (2026-07-07T12:26:06.335Z)

With redaction now a real slice, the tracker needs to stop serializing replay behind handoff proposal generation and show the shared proposal/provenance work separately. I?m restructuring the K rows while keeping the PR-sized shape.

153. assistant (2026-07-07T12:26:33.603Z)

The tracker now has separate slices for read-only handoff audit, proposal/Prediction trace plumbing, redaction v0, replay bundle, and replay ingest. I?m updating open questions and done criteria so those safety requirements don?t get lost later.

154. assistant (2026-07-07T12:26:54.600Z)

The done criteria now include the reviewer?s two external-boundary risks: unresolved provenance references must be rejected before candidates, and redaction must fail closed before egress. I?m adding the bundle ignore rule now, since that?s a tiny concrete safety improvement this PR can make immediately.

155. assistant (2026-07-07T12:27:00.390Z)

The `.gitignore` now protects future replay bundles even before the bundle command exists. I?m refreshing the handoff so the next session starts from the current review-addressed state, not the previous one.

156. assistant (2026-07-07T12:27:21.033Z)

I?m scanning for stale references from the old K numbering and old provenance wording now. These little renumbering ghosts are exactly how trackers become haunted later.

157. assistant (2026-07-07T12:27:27.839Z)

The scan caught a few stale K-number references from before the tracker split. I?m correcting those so the review response does not accidentally reintroduce ambiguity.

158. assistant (2026-07-07T12:27:46.992Z)

One handoff sentence was still a little muddled after renumbering; I?m smoothing that out before verification.

159. assistant (2026-07-07T12:27:55.254Z)

Caught one more: read-only handoff audit is K2 after the split, not K4. Fixing that now.

160. assistant (2026-07-07T12:28:08.425Z)

The K-numbering is now consistent in the handoff. I?m doing another stale-reference scan across the touched files before running tests.

161. assistant (2026-07-07T12:28:12.474Z)

No stale wording from the old plan remains, and whitespace is clean. I?m reviewing the final diff now for readability and then I?ll run the full verification set.

162. assistant (2026-07-07T12:28:20.252Z)

The diff looks coherent, but the loop-closure tracker still points P6 at the old K3 number. I?m updating that cross-link so the two trackers agree.

163. assistant (2026-07-07T12:28:36.342Z)

The loop tracker now agrees with the split K rows. I?m doing a final consistency scan before the verification suite.

164. assistant (2026-07-07T12:28:41.256Z)

The consistency scan found two stale row references inside the relation table and project-slug note. I?m fixing those last mismatches.

165. assistant (2026-07-07T12:28:52.850Z)

One more relation-table label moved in the split: project-page upserts are K3 now. Fixing that too.

166. assistant (2026-07-07T12:29:04.939Z)

The relation table now matches the tracker split. I?m running the final scan and then the full repo gates.

167. assistant (2026-07-07T12:29:08.513Z)

Stale-reference scan is clean, and ruff/mypy/whitespace are green. I?m running the full pytest coverage gate now.

168. assistant (2026-07-07T12:29:33.419Z)

All local gates are green: whitespace, ruff, mypy, and the full pytest coverage run. I?m committing the consolidated review response now, with explicit staging only.

169. assistant (2026-07-07T12:29:39.253Z)

Branch safety checks out: still on the PR branch, with only the PR?s own commits over `origin/main`. Staging the four touched files explicitly.

170. assistant (2026-07-07T12:29:45.026Z)

The consolidated review-response commit is created. I?m pushing it to PR #45 and then I?ll wait for CI before posting the reviewer-facing response.

171. assistant (2026-07-07T12:29:54.205Z)

The branch is pushed. I?m checking PR status and watching CI now.

172. assistant (2026-07-07T12:29:58.400Z)

PR #45 has the new commit and CI is queued. I?m waiting on the check before posting the response matrix.

173. assistant (2026-07-07T12:30:36.225Z)

CI passed on the new commit. I?m posting a detailed response to the consolidated review now, with the changed files and verification.

174. assistant (2026-07-07T12:31:23.188Z)

Reviewed the new PR comments and responded accordingly.

I pushed `1e5409e` to PR #45 and posted a detailed response comment: [issuecomment-4903812715](https://github.com/avidullu/agent-sessions/pull/45#issuecomment-4903812715).

What changed:
- Made replay redaction a first-class deliverable and added `.gitignore` coverage for `baseline/replay/bundles/`.

- Aligned trace/provenance fields to #19.
- Added ingest-side provenance-integrity requirements for `replay_of`, `markdown_path`, and `session_id`.
- Split shared proposal/`Prediction` trace plumbing into its own K-slice.
- Allowed read-only handoff audit to ship before schema-gated writes.
- Added deterministic handoff parsing limits, replay value kill criteria, and corrected tracker cross-links/K numbering.

Verification passed locally and in GitHub CI:
- `git diff --check`
- `ruff`
- `mypy`
- `pytest --cov` ? 408 passed, 96.72% coverage
- PR #45 CI passed

::git-stage{cwd="C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan"}
::git-commit{cwd="C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan"}
::git-push{cwd="C:/Users/avidu/Projects/Agent Sessions - knowledge-replay-design-plan" branch="codex/knowledge-replay-design-plan"}

175. user (2026-07-07T12:37:59.063Z)

ptal at the review comments and merge if the reviewwer has signalled the same. Follow up by picking up the items in the tracker in the order, sending PRs and then waiting for the review comments and addressing them till you get an LGTM and then move to the next item. Ensure the test coverage and documentations are up to date.

176. assistant (2026-07-07T12:38:14.429Z)

I?ll start by checking PR #45?s latest review state. If there?s an explicit approval/LGTM or equivalent merge signal, I?ll run the verified gate set, merge it, update my local base, and then begin the tracker items in order with one PR at a time.

177. assistant (2026-07-07T12:38:25.396Z)

I?m using the PR comment handler workflow again. I?ll rely on `gh` for this private repo and inspect both top-level review submissions and any inline review-thread state before deciding whether merge is allowed.

178. assistant (2026-07-07T12:38:46.743Z)

The latest review has a clear merge signal: ?ready to merge as the tracking doc.? There?s one non-blocking K2/K6 naming wrinkle the reviewer explicitly says can ride in the first K-slice PR, so I?m going to verify gates one more time, mark the draft ready, and merge #45 if everything stays green.

179. assistant (2026-07-07T12:38:51.535Z)

There are no inline review threads, and whitespace/ruff/mypy are clean. I?m running the full coverage gate now before merging.

180. assistant (2026-07-07T12:39:16.009Z)

The full local gate is green again: 408 tests, 96.72% coverage, and the repo?s 80% floor is met. I?m marking the draft PR ready and merging it now, since the reviewer explicitly signaled ?ready to merge.?

181. assistant (2026-07-07T12:39:34.762Z)

Marking ready succeeded, but the CLI merge path tried to touch the local `main` branch and hit the primary checkout that already owns it. I?ll merge the PR remotely via GitHub?s API instead, then delete the remote feature branch.

182. assistant (2026-07-07T12:39:44.227Z)

PR #45 is actually merged on GitHub; the failure was from local branch cleanup, not the remote merge. I?m fetching the new `origin/main` and checking whether the remote feature branch still exists.

183. assistant (2026-07-07T12:40:01.004Z)

PR #45 is merged as merge commit `fdf88a6`. The remote feature branch still exists, so I?m cleaning that up, then I?ll branch from fresh `origin/main` for K1 (`baseline/SCHEMA.md`) and carry the reviewer?s non-blocking K2/K6 naming clarification in that first implementation PR.

184. assistant (2026-07-07T12:40:15.343Z)

I?m creating a fresh worktree from `origin/main` for K1 rather than reusing the merged branch. That keeps the next PR based exactly on the remote main state after #45.

185. assistant (2026-07-07T12:40:33.365Z)

The K1 worktree is created at `C:\Users\avidu\Projects\Agent Sessions - baseline-schema-doc`. I?m doing the required pull there, then reading the merged tracker and current baseline bundle/scaffold code so the schema doc hooks into the right places.

186. assistant (2026-07-07T12:41:32.573Z)

I?ve got the thread state back after compaction. PR #45 was review-approved and is merged; I?m continuing on the first tracker item now in an isolated worktree so the shared checkout stays calm.

187. assistant (2026-07-07T12:41:44.360Z)

I?m refreshing the K1 worktree from `origin/main` before reading anything, per your repo rule, then I?ll make the schema/doc changes against the merged tracker state.

188. assistant (2026-07-07T12:41:53.329Z)

The K1 branch is current. I?m checking the existing baseline scaffold, bundle prompt code, and tests so the schema doc lands where the repo already expects this kind of contract.

189. assistant (2026-07-07T12:42:01.352Z)

The implementation hook is small: the bundle writer already builds one packet and prompt, while the scaffold owns baseline template files. I?m adding the schema to that scaffold and teaching bundle packets/prompts to point at it when present.

190. assistant (2026-07-07T12:42:26.539Z)

One subtle point from the review is worth carrying now: K2 should be the audit/report-only slice, while K6 owns persistent handoff indexes and page feeds. I?m folding that into the tracker text while I touch the K1 schema row.

191. assistant (2026-07-07T12:42:35.266Z)

I?m about to edit the scaffold/template layer, bundle prompt code, and the K tracker doc. The goal is K1 only: define the schema contract and make current bundle packets aware of it, without sneaking in handoff index writes that belong to K6.

192. assistant (2026-07-07T12:43:13.961Z)

The schema template is now part of scaffold generation. Next I?m wiring it through the public baseline module and the bundle code so tests can cover both the file contract and agent packet reference.

193. assistant (2026-07-07T12:43:18.841Z)

The bundle packet currently has proposal shape and evidence, but no schema contract. I?m adding a small optional schema payload that appears only when `baseline/SCHEMA.md` exists, which keeps older/test repositories working.

194. assistant (2026-07-07T12:43:41.051Z)

I?m adding the actual repository schema file now, matching the scaffold contract, then I?ll update the visible baseline README so reviewers can find it without knowing the command internals.

195. assistant (2026-07-07T12:44:19.583Z)

The proposal shape is still intentionally illustrative, but I?m adding the trace vocabulary there too so external proposal authors see the same #19-aligned fields as the schema and bundle prompt.

196. assistant (2026-07-07T12:44:38.088Z)

I?m updating the tracker carefully around the review note: K2 becomes a report-only audit, and K6 is where the persistent handoff index and knowledge feeds live. That keeps the execution order honest.

197. assistant (2026-07-07T12:45:03.648Z)

The docs now separate audit from indexing. I?m scanning for leftover wording so the tracker doesn?t imply the audit command secretly owns index writes.

198. assistant (2026-07-07T12:45:12.396Z)

I?m expanding tests around the new contract surface: scaffold includes `SCHEMA.md`, the template has the expected trace/marker language, and bundle output includes the schema reference when the file is present.

199. assistant (2026-07-07T12:45:16.038Z)

The temporary test repo fixture does not create a schema by default, so I?m adding explicit schema setup in the bundle test. That makes the ?when present? behavior crisp.

200. assistant (2026-07-07T12:45:29.801Z)

The first patch didn?t match the import ordering, so I?m applying the test edits in smaller chunks.

201. assistant (2026-07-07T12:45:44.344Z)

Now I?m adding the bundle-specific assertions: proposal schema exposes `trace`, prompt rendering mentions `baseline/SCHEMA.md`, and the written JSON packet embeds the contract when present.

202. assistant (2026-07-07T12:45:54.461Z)

The written-bundle test is getting one extra fixture file so it can prove the new packet field exists without changing repositories that have not scaffolded a schema yet.

203. assistant (2026-07-07T12:46:03.451Z)

I?m also updating the test-plan doc because the baseline and bundle test counts changed. Small bookkeeping, but it keeps the tracker from lying after the PR.

204. assistant (2026-07-07T12:46:13.956Z)

Before broad verification, I?m checking the project?s lint settings so the long schema template

doesn?t trip a mechanical line-length rule inside Python strings.

205. assistant (2026-07-07T12:46:21.746Z)

The line-length cap is 120. I?m going to run the focused baseline tests and ruff now; if the schema template needs wrapping, this will catch it quickly.

206. assistant (2026-07-07T12:46:31.205Z)

The focused checks are green: 87 targeted tests passed and ruff is clean for the touched Python files. I?m getting exact test counts now so the docs update is precise.

207. assistant (2026-07-07T12:46:38.882Z)

The exact current counts are 70 baseline tests and 17 bundle tests. I?ll update those rows now, then the full-suite row after the final coverage run.

208. assistant (2026-07-07T12:46:50.715Z)

Since the design tracker is no longer just a draft review doc after #45 merged and K1 started, I?m switching its status language to active execution in the docs index.

209. assistant (2026-07-07T12:47:05.074Z)

The main docs are now aligned to active execution. I?m checking the diff shape before running the full gate, mostly to catch accidental scope creep.

210. assistant (2026-07-07T12:47:17.454Z)

The diff is scoped to K1 docs, bundle/schema plumbing, and tests. I?m running the full pre-PR gate now: whitespace, ruff, mypy, and the full coverage suite against the repo?s 80% floor.

211. assistant (2026-07-07T12:47:41.095Z)

The full gate passed: 410 tests, 96.75% coverage, plus ruff, mypy, and diff-check. I?m recording that exact verification in the test plan before committing.

212. assistant (2026-07-07T12:48:03.026Z)

Only the test-plan doc changed after the full suite, so I?m doing a final whitespace check before creating the first K1 commit.

213. assistant (2026-07-07T12:48:16.794Z)

The workspace is ready to commit. I?m doing the branch-safety checks now, then I?ll stage only the explicit K1 paths.

214. assistant (2026-07-07T12:48:21.518Z)

Branch is `codex/baseline-schema-doc` on the #45 merge commit, with only the K1 files dirty. Staging those explicit paths now.

215. assistant (2026-07-07T12:48:30.220Z)

Staging succeeded. I?m doing the last pre-commit branch check and confirming no unrelated files slipped in.

216. assistant (2026-07-07T12:48:34.853Z)

The staged set is clean and `origin/main..HEAD` is empty before the commit, so I?m committing the K1 slice.

217. assistant (2026-07-07T12:48:41.871Z)

Commit `fe08575` is created. I?m verifying the branch contains only that commit above `origin/main`, then I?ll push and open the PR.

218. assistant (2026-07-07T12:48:45.747Z)

I?m using the GitHub publish workflow skill for the PR creation step so the branch/PR handoff follows the repo?s expected discipline.

219. assistant (2026-07-07T12:48:59.121Z)

`gh` is the expected path here, and this PR should be reviewable rather than draft because your workflow asks me to wait for review comments and LGTM. I?m checking auth and then pushing the branch.

220. assistant (2026-07-07T12:49:02.453Z)

GitHub auth is good. Pushing `codex/baseline-schema-doc` now.

221. assistant (2026-07-07T12:49:13.417Z)

The branch is on GitHub. I?m confirming the repo default branch and opening the review PR with the validation details in the body.

222. assistant (2026-07-07T12:49:21.748Z)

Opening PR now. After GitHub assigns a number, I?ll update the tracker/handoff with that concrete PR link instead of leaving placeholders.

223. assistant (2026-07-07T12:49:36.786Z)

PR #46 is open. I?m updating the tracker and handoff with the real PR number and current review state, then I?ll push that bookkeeping commit onto the same PR.

224. assistant (2026-07-07T12:49:45.696Z)

The project handoff still described #45 as a draft, so I?m making it reflect the shipped state: #45 merged, #46 is the active K1 PR, and K2 is next only after review/merge.

225. assistant (2026-07-07T12:50:04.791Z)

The PR number is now in the tracker and handoff. This second commit is docs-only; I?m running the whitespace check and branch-safety checks before staging just those files.

226. assistant (2026-07-07T12:50:07.601Z)

Only the two bookkeeping docs are dirty. I?m staging those explicit files and committing them onto PR #46.

227. assistant (2026-07-07T12:50:16.166Z)

Staged. One more check that the branch contains only my K1 commits above `origin/main`, then the docs commit goes up.

228. assistant (2026-07-07T12:50:19.147Z)

The staged set is exactly the tracker and handoff, with the single K1 implementation commit already on the branch. Committing the PR-number update now.

229. assistant (2026-07-07T12:50:23.662Z)

Docs commit `6636a3f` is ready; pushing it to PR #46 now.

230. assistant (2026-07-07T12:50:33.965Z)

PR #46 now has the tracker/handoff update too. I?m switching back to the PR-comment workflow and checking review threads plus CI state before deciding whether there?s anything to address.

231. assistant (2026-07-07T12:50:52.369Z)

I?m pulling both the flat PR state and the thread-aware review data. That distinction matters because unresolved inline threads are where actionable reviewer work usually hides.

232. assistant (2026-07-07T12:50:58.114Z)

PR #46 is open and ready for review; no comments or review threads yet. CI has started and is still pending, so I?m waiting for that before doing the next review poll.

233. assistant (2026-07-07T12:51:20.392Z)

CI passed. I?m polling review threads and top-level comments again now that the check finished.

234. assistant (2026-07-07T12:51:33.893Z)

The branch is clean, CI passed, and there are still no review comments or approvals. I?ll give the reviewer window another poll cycle before I stop; I won?t merge or move to K2 without LGTM.

235. assistant (2026-07-07T12:52:11.609Z)

Still no flat comments or reviews. I?m checking thread-aware inline comments once more, since those can be missed by the simpler PR view.

236. assistant (2026-07-07T12:52:23.017Z)

There are no inline threads, top-level comments, or reviews on #46 yet. I?m marking the implementation PR step complete and the review wait as the active blocker before I hand you the current state.

237. assistant (2026-07-07T12:52:47.415Z)

Merged PR #45 after the reviewer signaled it was ready.

Opened K1 PR #46: [Add baseline schema contract](https://github.com/avidullu/agent-sessions/pull/46). It adds `baseline/SCHEMA.md`, wires it into scaffold and `baseline bundle` packets/prompts, updates the tracker/handoff/docs, and carries the K2/K6 audit-vs-index cleanup from review.

Verified locally:
`git diff --check`, `ruff check .`, `mypy agent_sessions tools`, and full coverage suite: `410 passed`, `96.75%` coverage. GitHub CI is also passing.

I checked PR #46 comments, reviews, and inline review threads after CI passed. There are no comments or LGTM yet, so I did not merge it or move on to K2.

::git-create-branch{cwd="C:/Users/avidu/Projects/Agent Sessions - baseline-schema-doc" branch="codex/baseline-schema-doc"}
::git-stage{cwd="C:/Users/avidu/Projects/Agent Sessions - baseline-schema-doc"}
::git-commit{cwd="C:/Users/avidu/Projects/Agent Sessions - baseline-schema-doc"}
::git-push{cwd="C:/Users/avidu/Projects/Agent Sessions - baseline-schema-doc" branch="codex/baseline-schema-doc"}
::git-create-pr{cwd="C:/Users/avidu/Projects/Agent Sessions - baseline-schema-doc" branch="codex/baseline-schema-doc" url="https://github.com/avidullu/agent-sessions/pull/46" isDraft=false}

238. user (2026-07-07T12:52:47.900Z)

please increase your polling duration to 10 minutes

239. assistant (2026-07-07T12:53:01.032Z)

I?ll poll PR #46 for a full 10 minutes before calling the wait. I?m using the GitHub review-comment workflow again, and I?ll still only merge if an actual approval/LGTM appears.

240. assistant (2026-07-07T12:53:07.685Z)

Before polling, I?m refreshing the PR worktree with a fast-forward-only pull so any local view of the branch matches GitHub.

241. assistant (2026-07-07T12:53:22.720Z)

Starting the 10-minute watch now. I?ll check both review state and inline threads on each pass, not just the top-level PR page.

242. assistant (2026-07-07T12:54:18.139Z)

Poll 1 of 20: CI is green, merge state clean, and there are still no reviews, comments, or unresolved inline threads. Continuing the 10-minute watch.

243. assistant (2026-07-07T12:54:28.220Z)

I?m going to keep the pass output concise from here: decision, comment/thread counts, review states, and CI. If a review arrives, I?ll break out of watch mode and handle it.

244. assistant (2026-07-07T12:55:12.033Z)

Poll 2 found a new review state: `COMMENTED`. I?m pausing the watch to inspect the actual review text and any inline context, then I?ll address or respond accordingly.

245. assistant (2026-07-07T12:55:32.020Z)

The review is a merge-ready LGTM posted as a comment because GitHub blocks self-approval. The reviewer explicitly marked the follow-ups non-blocking, so I?m going to rerun the local merge gate on the exact branch tip and merge if it stays green.

246. assistant (2026-07-07T12:55:40.609Z)

Branch tip is up to date and CI is still green. Running the local gate now: diff check, ruff, mypy, and full coverage suite.

247. assistant (2026-07-07T12:56:06.554Z)

The local gate is green on the reviewed tip: diff-check clean, ruff clean, mypy clean, and `410 passed` with `96.75%` coverage. I?m merging PR #46 now since the reviewer?s comment says ?merge-ready.?

248. assistant (2026-07-07T12:56:17.959Z)

The merge command hit the same local worktree quirk as PR #45: GitHub may have merged it, but `gh` failed while trying to touch the local `main` branch that belongs to another worktree. I?m verifying the remote PR state before doing anything else.

249. assistant (2026-07-07T12:56:23.691Z)

Confirmed: PR #46 is merged at remote merge commit `9450f1f`. I?m checking whether the remote feature branch still needs cleanup.

250. assistant (2026-07-07T12:56:30.817Z)

The remote feature branch is still there because the local merge helper bailed during cleanup. Deleting that remote branch explicitly now.

251. assistant (2026-07-07T12:56:42.765Z)

K1 is now merged and the remote branch is cleaned up. I?m moving to the next tracker item, K2, from fresh `origin/main` in a new worktree.

252. assistant (2026-07-07T12:56:48.388Z)

Per the branching rule, I?ll branch K2 directly from `origin/main`, not from the just-merged worktree branch.

253. assistant (2026-07-07T12:57:03.189Z)

The K2 worktree is created at the merged K1 commit. I?m doing the required fast-forward pull there before reading code.

254. assistant (2026-07-07T12:57:14.932Z)

I?m ramping on the K2 surface now: CLI shape, archive index structure, handoff conventions, and the tracker/schema language. This slice should only write an audit report, not an index or project-page/proposal output.

255. assistant (2026-07-07T12:57:38.243Z)

The K2 shape is pretty clear now. I?m going to add a focused `baseline_handoffs` module plus a nested CLI command: `python tools/agent_archive.py baseline handoffs audit`. Its default write target will be only `baseline/handoffs/audit.md`; no index, page, or proposal writes.

256. assistant (2026-07-07T12:57:43.402Z)

I?m reading the CLI tests before editing so the nested command follows existing argparse coverage style.

257. assistant (2026-07-07T12:58:31.454Z)

I?m making the K2 command report useful coverage and freshness, but keeping its data model ephemeral. Persistent normalized handoff records stay deliberately out of this PR.

258. assistant (2026-07-07T13:00:06.635Z)

The report-only module is in place. I?m wiring the nested CLI command next, with `--dry-run`, `--output`, `--stale-days`, and a scan limit for large archives.

259. assistant (2026-07-07T13:00:20.649Z)

I?m adding focused tests around the new audit behavior: repo handoff parsing, archive signal detection, dry-run/no-index writes, and CLI dispatch.

260. assistant (2026-07-07T13:00:48.500Z)

The audit unit tests are in. I?m adding the CLI coverage and import-hygiene check so this new module stays in the baseline split contract.

261. assistant (2026-07-07T13:01:06.538Z)

I?m doing a focused pass now; this should flush out any line wrapping, mypy-ish typing issues, or parser mistakes before I touch the tracker/docs.

262. assistant (2026-07-07T13:01:22.654Z)

Focused checks passed. I?m going to run the new command for real next so the PR includes the K2 report artifact itself, then I?ll update the tracker and test-plan docs.

263. assistant (2026-07-07T13:01:31.746Z)

The wrapper script doesn?t put this worktree root on `PYTHONPATH` when run directly in this checkout, so I?m using the module entry with `PYTHONPATH` set for the real audit generation.

264. assistant (2026-07-07T13:01:56.831Z)

That exposed a small wrapper bug: `python tools/agent_archive.py ...` needs `PYTHONPATH` today, even though docs use that wrapper. Since K2 documents this command path, I?m going to fix the wrapper import path and cover it with a tiny subprocess test.

265. assistant (2026-07-07T13:02:01.957Z)

The wrapper now adds the repo root to `sys.path` before importing the package. I?m adding a regression test so the documented script path stays usable.

266. assistant (2026-07-07T13:02:12.943Z)

I?m rerunning the audit through the plain wrapper path now, both as verification and to refresh the generated report after the wrapper fix.

267. assistant (2026-07-07T13:02:37.257Z)

The real audit report is generated and confirms the K2 boundaries: it found the repo handoff, scanned 3,114 archive records, found 946 archive handoff candidates, and wrote only `baseline/handoffs/audit.md`. I?m updating the docs and tracker around that.

268. assistant (2026-07-07T13:02:43.983Z)

I?m updating the docs in three places: the active project tracker, the baseline overview, and the test-plan inventory. Also refreshing the handoff so a resumed session starts from the merged K1 plus active K2 state.

269. assistant (2026-07-07T13:02:59.595Z)

The tracker now marks K1 done and K2 active. I?m updating the test-plan counts after adding the handoff-audit and wrapper tests; the exact final count will be confirmed by the full suite.

270. assistant (2026-07-07T13:03:10.511Z)

The handoff is next. I?m keeping it factual: K1 merged, K2 branch/worktree active, report generated, and reviewer?s optional follow-ups carried forward without pretending they?re shipped.

271. assistant (2026-07-07T13:03:37.420Z)

While I?m on K2, I?m taking the reviewer?s non-blocking K1 hardening that fits cleanly here: schema drift guard, absent-schema coverage, and the calibration artifact row clarification. They?re small, and they reduce future review churn.

272. assistant (2026-07-07T13:03:42.559Z)

The committed schema got the calibration artifact row. I?m making the scaffold template match exactly so the golden drift test can protect them going forward.

273. assistant (2026-07-07T13:03:49.630Z)

Now I?m adding the reviewer-requested guard tests: committed schema equals scaffold template, and bundle prompt generation stays clean when no schema file exists.

274. assistant (2026-07-07T13:04:10.199Z)

Those optional hardening tests are added. I?m counting the updated test files so the test plan gets exact numbers after this round.

275. assistant (2026-07-07T13:04:16.991Z)

The exact touched-file counts are: baseline 71, bundle 19, CLI 45, handoffs 7, wrapper 1. I?m correcting the test-plan table before the full-suite number lands.

276. assistant (2026-07-07T13:04:28.960Z)

I?m rerunning the focused K2 and hardening tests now. This checks the report-only audit, wrapper fix, schema drift guard, and CLI parser paths together.

277. assistant (2026-07-07T13:04:37.505Z)

Focused tests passed, and ruff caught the expected `sys.path` wrapper pattern as E402. I?m adding the narrow noqa on that import, since the import order is intentional.

278. assistant (2026-07-07T13:04:41.805Z)

I?m rerunning ruff on the touched paths after that wrapper import annotation.

279. assistant (2026-07-07T13:04:52.253Z)

Focused ruff is clean now. I?m reviewing the K2 diff and checking that no prohibited outputs like `index.jsonl` or proposals were created.

280. assistant (2026-07-07T13:05:02.038Z)

The generated output boundary looks right: `baseline/handoffs/audit.md` exists, `baseline/handoffs/index.jsonl` does not, and there are no new proposal JSON files. I?m running the full local gate now.

281. assistant (2026-07-07T13:05:22.220Z)

Full tests passed with `424 passed` and 95.91% coverage; ruff/diff-check are clean. Mypy found a small variable-name reuse issue in the gaps renderer, so I?m fixing that and rerunning the gate.

282. assistant (2026-07-07T13:05:28.419Z)

The mypy issue is just the same loop variable used for strings and repo records. Renaming it; tiny, but mypy is right to be fussy here.

283. assistant (2026-07-07T13:05:38.320Z)

I?m rerunning mypy and the focused handoff tests after the rename, then I?ll update the final coverage line in the test plan.

284. assistant (2026-07-07T13:05:42.514Z)

Mypy is clean now. I?m recording the actual full-suite result from the last run: 424 tests and 95.91% coverage.

285. assistant (2026-07-07T13:05:53.159Z)

The documentation now matches the measured suite. I?m running the final pre-commit gate again from scratch: diff check, ruff, mypy, and full coverage.

286. assistant (2026-07-07T13:06:14.914Z)

Final local gate is green: diff-check, ruff, mypy, and `424 passed` with 95.91% coverage. I?m checking the final file set before staging explicit paths.

287. assistant (2026-07-07T13:06:20.209Z)

One handoff note still reads like those K1 follow-ups are pending, but I actually implemented

them in this branch. I?m tightening that wording so the handoff stays honest.

288. assistant (2026-07-07T13:06:34.634Z)

That was docs-only, but I?m doing one last full gate after it so the PR body can honestly say the exact branch tip was checked.

289. assistant (2026-07-07T13:06:58.804Z)

The branch tip is fully verified. I?m fetching `origin` and doing the pre-commit branch checks before staging only the K2 files.

290. assistant (2026-07-07T13:07:07.687Z)

The K2 branch is cleanly based on `origin/main` at the PR #46 merge commit, and there are no commits above main yet. Staging the explicit K2 paths now.

291. assistant (2026-07-07T13:07:17.038Z)

Staging is done. I?m confirming the staged set and branch ancestry before the commit.

292. assistant (2026-07-07T13:07:20.706Z)

Everything staged belongs to K2 and `origin/main..HEAD` is empty before committing. Creating the K2 commit now.

293. assistant (2026-07-07T13:07:30.103Z)

Commit `0653f0c` is created. I?m verifying it?s the only branch commit, then I?ll push and open the K2 PR.

294. assistant (2026-07-07T13:07:33.851Z)

The branch contains only the K2 commit and the worktree is clean. Pushing now.

295. assistant (2026-07-07T13:07:40.458Z)

The branch is on GitHub. I?m opening the K2 PR as ready for review with the verification and report boundary spelled out.

296. assistant (2026-07-07T13:07:55.375Z)

PR #47 is open. I?m adding the concrete PR number to the tracker and handoff, then I?ll push that bookkeeping commit to the same PR.

297. assistant (2026-07-07T13:08:11.367Z)

The tracker now points at #47. This is a docs-only follow-up commit; I?m checking it and pushing it onto the PR.

298. assistant (2026-07-07T13:08:14.515Z)

Only the tracker and handoff are dirty. Staging those explicit files now.

299. assistant (2026-07-07T13:08:22.214Z)

Staged. I?m confirming no extra paths slipped in, then committing the PR-number update.

300. assistant (2026-07-07T13:08:25.524Z)

The staged set is just the handoff and tracker. Committing the #47 references now.

301. assistant (2026-07-07T13:08:31.879Z)

Pushing the tracker-reference commit to PR #47 now.

302. assistant (2026-07-07T13:08:41.843Z)

PR #47 now includes the tracker reference. I?m checking CI and review state, then I?ll use the longer 10-minute polling window for comments/LGTM.

303. assistant (2026-07-07T13:08:46.889Z)

PR #47 is open and CI is queued. I?ll wait for CI first; once it?s green, I?ll do the 10-minute review/comment polling loop unless a review arrives earlier.

304. assistant (2026-07-07T13:09:40.427Z)

CI is green. Starting the 10-minute review/comment poll for PR #47 now; it will stop early if any review, top-level comment, or inline thread appears.

305. assistant (2026-07-07T13:12:09.382Z)

The 10-minute poll stopped early at poll 5 because the reviewer posted a merge-ready LGTM comment. The optional items are explicitly non-blocking and can ride K6, so I?m rerunning the merge gate on the exact PR tip and then merging.

306. assistant (2026-07-07T13:12:18.131Z)

The branch is up to date and CI is green. Running local diff-check, ruff, mypy, and full coverage one more time before merge.

307. assistant (2026-07-07T13:12:35.238Z)

The merge gate is green on #47?s exact tip: diff-check, ruff, mypy, full coverage, and CI all pass. Merging PR #47 now.

308. assistant (2026-07-07T13:12:48.185Z)

The local `main` worktree collision happened again during merge cleanup. As before, I?m verifying the remote state before taking cleanup steps.

309. assistant (2026-07-07T13:12:54.368Z)

PR #47 is merged at remote commit `59dbd65`. The helper did not delete the branch because of the local worktree collision, so I?m checking and cleaning up the remote branch manually.

310. assistant (2026-07-07T13:14:08.962Z)

Done. I?ll use a 10-minute polling window for each PR review cycle, and I?ll keep stopping early when a reviewer posts a clear signal so the work can keep moving.

311. assistant (2026-07-07T13:14:14.526Z)

I?m going to re-anchor the local checkouts to remote state before touching the next tracker item. First I?ll verify the just-merged K2 branch cleanup, then branch K3 from `origin/main` so it starts from the merged truth.

312. assistant (2026-07-07T13:14:30.204Z)

The primary checkout has an uncommitted `SESSION_HANDOFF.md`, so the safe pull correctly refused to overwrite it. I?ll leave that WIP alone and work from fresh worktrees based explicitly on `origin/main`.

313. assistant (2026-07-07T13:14:36.474Z)

K2 is merged, but the remote topic branch is still hanging around because the merge command?s local checkout cleanup hit the shared-worktree guard. I?m deleting that remote branch now.

314. assistant (2026-07-07T13:14:45.561Z)

Branch cleanup is done. Next up is K3 from the tracker, so I?m creating a clean isolated worktree from `origin/main` and will read the freshly merged docs/tests there before editing.

315. assistant (2026-07-07T13:15:01.746Z)

The K3 worktree is ready at `origin/main` including the K2 merge. I?m doing the required pull check inside it, then I?ll map the upsert helpers and project-page code before changing anything.

316. assistant (2026-07-07T13:15:13.740Z)

I?m reading K3?s local neighborhood now: the tracker row, the existing promoted-content upsert helper, and the baseline/project-page generation path. That should show whether this is a reuse job or a small extraction/refactor.

317. assistant (2026-07-07T13:15:43.690Z)

K3 is confirmed as a marker-block reuse slice, and the repo already has a dedicated `baseline_promote.py` helper. I?m narrowing in on that helper and the tests around it before deciding the smallest extension.

318. assistant (2026-07-07T13:15:57.144Z)

The existing helper is deliberately generic enough in behavior, but its naming and header defaults are still guardrail-oriented. I?m going to make it project-page friendly without introducing a second marker system.

319. assistant (2026-07-07T13:16:17.217Z)

Before editing K3, I?m also checking the live issue text for #26 so the helper lands against the current ask, not just the tracker?s memory of it.

320. assistant (2026-07-07T13:17:09.221Z)

The live #26 acceptance criteria matches the local tracker: preserve human prose, reuse `baseline:begin/end`, and prove project pages can be updated twice without duplicate generated blocks. I?m going to add that as a small infrastructure slice with tests.

321. assistant (2026-07-07T13:17:35.405Z)

I?m about to edit `baseline_promote.py` and its public re-export surface, then add focused tests in `tests/test_baseline.py`. The docs/tracker updates will follow after the behavior is green.

322. assistant (2026-07-07T13:18:07.563Z)

The marker upsert is now factored under one shared path, with project-page wrappers using the same `baseline:begin/end` grammar. I?m wiring those helpers through the public baseline API and then adding regression tests.

323. assistant (2026-07-07T13:18:14.680Z)

I?m adding tests that simulate exactly the K3 contract: a scaffolded project page with human prose, a generated block inserted once, then the same block updated on a later run.

324. assistant (2026-07-07T13:18:35.018Z)

The core tests are in place. I?m running the targeted baseline test subset now, because it will catch both the old global promotion behavior and the new project-page wrapper.

325. assistant (2026-07-07T13:18:47.725Z)

Targeted tests pass: `test_baseline.py` is now 74 tests, including the project-page upsert path. I?m updating the schema/tracker docs next so K3 is documented as a shared helper slice rather than an ad hoc new writer.

326. assistant (2026-07-07T13:18:58.261Z)

The implementation diff looks compact, with one formatting cleanup I?ll handle. Now I?m patching the schema text in both the committed `baseline/SCHEMA.md` and its scaffold template so the golden drift test stays honest.

327. assistant (2026-07-07T13:19:04.904Z)

The schema already says ?no second marker family?; I?m adding the concrete K3 helpers there so future K6/K7 writers know exactly which path to use.

328. assistant (2026-07-07T13:19:08.670Z)

The schema template has to move in lockstep with `baseline/SCHEMA.md`, so I?m patching both copies together and keeping the golden test meaningful.

329. assistant (2026-07-07T13:19:25.365Z)

I?m updating the tracker to reflect the real state after K2 merged and K3 began. I?ll fill in the actual K3 PR number after GitHub assigns it.

330. assistant (2026-07-07T13:19:35.469Z)

The tracker now shows K2 done and K3 active. I?m adding the same project-page helper contract to the engineering overview so future maintainers see it outside the long tracker too.

331. assistant (2026-07-07T13:19:44.045Z)

The code/docs slice is coherent enough for a first verification pass. I?m running formatter/lint/type/test checks now, then I?ll update the test-plan numbers from the actual run.

332. assistant (2026-07-07T13:19:49.400Z)

Lint and type checks are clean. I?m running the full coverage suite now; this is the merge-gate baseline I?ll use again before merge if the reviewer approves.

333. assistant (2026-07-07T13:20:11.959Z)

Full suite is green at 427 passed, but coverage dipped from the K2 branch?s 95.91% to 95.75% because the new helper has a few untested branches. I?m tightening that now so the coverage gate is not merely ?above floor? but non-regressing.

334. assistant (2026-07-07T13:20:32.009Z)

The uncovered spots are useful edge cases, not busywork: empty updates, extra trace fields, and ID-filtered promotion. I?m adding those tests and trimming an unnecessary private-helper branch.

335. assistant (2026-07-07T13:20:38.182Z)

I?m adding those edge tests in the existing baseline test class so they travel with the helper behavior.

336. assistant (2026-07-07T13:20:49.945Z)

I found the existing promote test helper, so the no-op promote case will be cheap to cover too. That should recover the non-regression margin nicely.

337. assistant (2026-07-07T13:21:10.265Z)

Those edge tests are patched in. I?ll rerun the targeted baseline tests first, then the full gate again if they?re clean.

338. assistant (2026-07-07T13:21:17.494Z)

Targeted tests and Ruff are clean with 77 baseline tests now. I?m rerunning the full coverage suite to check the non-regression threshold.

339. assistant (2026-07-07T13:21:37.579Z)

Great, the full suite is now 430 passed with 96.16% coverage, so it beats the previous 95.91% baseline. I?m updating the test plan with the actual numbers.

340. assistant (2026-07-07T13:21:51.307Z)

Docs now reflect the real test count and coverage. I?m doing a self-review pass on the diff before committing, including whitespace checks and the branch/head safety checks from the repo agreement.

341. assistant (2026-07-07T13:22:03.545Z)

The self-review diff is in good shape: no whitespace errors, no second marker family, and `baseline_promote.py` now has 100% coverage. I?m verifying branch ancestry before staging explicit paths and committing.

342. assistant (2026-07-07T13:22:06.872Z)

Branch ancestry is clean: K3 is directly on the merged K2 `origin/main`, with no commits yet. I?m staging only the K3 files now.

343. assistant (2026-07-07T13:22:12.506Z)

Only the intended eight files are staged. I?m committing the K3 helper slice now.

344. assistant (2026-07-07T13:22:20.555Z)

The K3 commit is in place. I?m doing the pre-push ancestry check now, then I?ll push and open the PR for review.

345. assistant (2026-07-07T13:22:23.387Z)

The branch contains only the K3 commit. Pushing it now.

346. assistant (2026-07-07T13:22:31.726Z)

I?m using the GitHub PR publishing workflow here, so I?m checking its local instructions before opening the PR.

347. assistant (2026-07-07T13:22:43.626Z)

The branch is already pushed and the checks have run. I?m confirming `gh` auth quickly, then I?ll open this as ready for review rather than draft, matching the ongoing tracker workflow.

348. assistant (2026-07-07T13:22:51.915Z)

Opening the K3 PR now with the verification notes and the scope boundary called out: helpers only here, K6 still owns the actual handoff index/project-page feed.

349. assistant (2026-07-07T13:23:05.845Z)

PR #48 is open. I?m making the follow-up tracker/handoff update so a fresh session starts from

the current reality, then I?ll push that small docs commit into the same PR.

350. assistant (2026-07-07T13:23:12.261Z)

The tracker now points at PR #48. I?m refreshing the session handoff with the same checkpoint and K3 decision.

351. assistant (2026-07-07T13:23:29.657Z)

The handoff now reflects K2 merged and K3 PR #48 open. I?m committing this metadata update separately so the implementation commit stays tidy.

352. assistant (2026-07-07T13:23:33.898Z)

Whitespace is clean and only the tracker/handoff changed. I?m staging those two paths explicitly and committing.

353. assistant (2026-07-07T13:23:41.690Z)

The tracker commit is made. I?m rechecking the branch range and then pushing the update to PR #48.

354. assistant (2026-07-07T13:23:44.372Z)

The branch range contains only the two K3 commits. Pushing the tracker update now.

355. assistant (2026-07-07T13:24:05.019Z)

PR #48 is updated. I?m starting the 10-minute review polling window now; I?ll also watch CI because reviewer comments often arrive after checks settle.

356. assistant (2026-07-07T13:24:15.891Z)

Initial PR state: no comments yet, CI pending on commit `0ea662c`. I?ll check again in about 30 seconds.

357. assistant (2026-07-07T13:24:51.842Z)

CI is green now and there are still no review comments. I?ll keep the 10-minute polling window running instead of moving to K4 early.

358. assistant (2026-07-07T13:25:27.505Z)

Still quiet on PR #48 after the next check. I?m continuing the poll window.

359. assistant (2026-07-07T13:26:06.047Z)

No review activity yet. The branch head remains `0ea662c`, so the reviewer will be seeing the latest pushed docs update.

360. assistant (2026-07-07T13:26:42.671Z)

Still no comments or reviews on #48. I?m checking again in the same 30-second cadence.

361. assistant (2026-07-07T13:27:19.396Z)

Nothing new yet. The good news is the checks stayed green; the only thing we?re waiting on is reviewer signal.

362. assistant (2026-07-07T13:27:54.588Z)

PR #48 is still open with no comments. I?ll keep the 10-minute polling window going.

363. assistant (2026-07-07T13:28:30.105Z)

No review activity yet. I?m at roughly the middle stretch of the 10-minute watch and still holding K4 until this PR gets a signal.

364. assistant (2026-07-07T13:29:05.128Z)

Still quiet on the PR. I?ll keep polling; if an LGTM lands, I?ll rerun the full local gate before merging.

365. assistant (2026-07-07T13:29:40.833Z)

No comments yet. CI remains already passed, so the reviewer has a clean PR to inspect.

366. assistant (2026-07-07T13:30:17.274Z)

Still no review signal. I?m continuing through the full 10-minute duration you asked for.

367. assistant (2026-07-07T13:30:53.296Z)

No update on #48 yet. I?ll do another check in 30 seconds.

368. assistant (2026-07-07T13:31:29.564Z)

Still no comments or formal review. I?m staying parked on K3 as requested, rather than starting K4 on top of an unreviewed branch.

369. assistant (2026-07-07T13:32:04.969Z)

No reviewer activity yet. The PR is still open and current.

370. assistant (2026-07-07T13:32:40.321Z)

Still quiet. I?m entering the last few minutes of the 10-minute polling window.

371. assistant (2026-07-07T13:33:15.606Z)

No comments yet. I?ll continue polling until the window completes, then I?ll report status cleanly.

372. assistant (2026-07-07T13:33:57.059Z)

Still no review signal on #48. The branch head is unchanged and CI already passed.

373. assistant (2026-07-07T13:34:33.237Z)

No update yet. Two-ish checks left in this polling window.

374. assistant (2026-07-07T13:35:09.193Z)

Still quiet on the PR. I?ll do the final checks and then pause on K3 until a reviewer responds.

375. assistant (2026-07-07T13:35:45.038Z)

No comments yet on the penultimate poll. One final check for the 10-minute window.

376. assistant (2026-07-07T13:36:52.889Z)

Set the PR polling window to 10 minutes and used it for PR #48.

K3 is now open: [PR #48](https://github.com/avidullu/agent-sessions/pull/48). CI is green, and I also verified locally:

`ruff check .` clean
`mypy agent_sessions tools` clean
`pytest --cov=agent_sessions --cov-report=term-missing` -> 430 passed, 96.16% coverage

The 10-minute poll completed with no comments or reviews yet, so I did not merge and did not start K4. I?ll keep K3 parked until reviewer signal/LGTM lands.

::git-create-branch{cwd="C:/Users/avidu/Projects/Agent Sessions - baseline-schema-doc" branch="codex/baseline-schema-doc"}
::git-stage{cwd="C:/Users/avidu/Projects/Agent Sessions - baseline-schema-doc"}
::git-commit{cwd="C:/Users/avidu/Projects/Agent Sessions - baseline-schema-doc"}
::git-push{cwd="C:/Users/avidu/Projects/Agent Sessions - baseline-schema-doc" branch="codex/baseline-schema-doc"}
::git-create-pr{cwd="C:/Users/avidu/Projects/Agent Sessions - baseline-schema-doc" branch="codex/baseline-schema-doc" url="https://github.com/avidullu/agent-sessions/pull/46" isDraft=false}
::git-create-branch{cwd="C:/Users/avidu/Projects/Agent Sessions - handoffs-audit" branch="codex/handoffs-audit"}
::git-stage{cwd="C:/Users/avidu/Projects/Agent Sessions - handoffs-audit"}
::git-commit{cwd="C:/Users/avidu/Projects/Agent Sessions - handoffs-audit"}
::git-push{cwd="C:/Users/avidu/Projects/Agent Sessions - handoffs-audit" branch="codex/handoffs-audit"}
::git-create-pr{cwd="C:/Users/avidu/Projects/Agent Sessions - handoffs-audit" branch="codex/handoffs-audit" url="https://github.com/avidullu/agent-sessions/pull/47" isDraft=false}
::git-create-branch{cwd="C:/Users/avidu/Projects/Agent Sessions - project-page-upserts" branch="codex/project-page-upserts"}
::git-stage{cwd="C:/Users/avidu/Projects/Agent Sessions - project-page-upserts"}
::git-commit{cwd="C:/Users/avidu/Projects/Agent Sessions - project-page-upserts"}
::git-push{cwd="C:/Users/avidu/Projects/Agent Sessions - project-page-upserts" branch="codex/project-page-upserts"}
::git-create-pr{cwd="C:/Users/avidu/Projects/Agent Sessions - project-page-upserts" branch="codex/project-page-upserts" url="https://github.com/avidullu/agent-sessions/pull/48" isDraft=false}

377. user (2026-07-07T13:36:54.810Z)

Please do a few more polls the review just picked up

378. assistant (2026-07-07T13:37:09.646Z)

Using the GitHub PR-comment workflow so I can catch both top-level reviews and inline threads cleanly. I?ll poll #48 a few more times now and stop early if review comments or an LGTM appear.

379. assistant (2026-07-07T13:37:17.013Z)

I?ve got the PR-comment workflow loaded. I?m refreshing the K3 worktree first, then I?ll check both flat PR reviews and thread-aware review data.

380. assistant (2026-07-07T13:37:26.826Z)

The branch is clean and up to date, and `gh` auth is good. I?m pulling the thread-aware review state now so inline comments do not get missed.

381. assistant (2026-07-07T13:37:33.154Z)

Flat PR metadata still shows no submitted review, but I?m going one level deeper with the bundled GraphQL helper in case there are pending/unresolved inline threads.

382. assistant (2026-07-07T13:37:50.913Z)

No visible review yet, so I?m doing a few more 30-second polls now. If the review is still being drafted, GitHub won?t expose it until it?s submitted, but we?ll catch it as soon as it lands.

383. assistant (2026-07-07T13:38:51.684Z)

The review landed on the first extra poll: clear LGTM / merge-ready, with three optional non-blocking follow-ups. I?m going to honor the merge gate now: rerun checks on the exact PR tip before merging.

384. assistant (2026-07-07T13:38:59.418Z)

The PR branch is clean, CI is green, and the branch range has only the two K3 commits. Running the full local gate now.

385. assistant (2026-07-07T13:39:03.633Z)

Whitespace, Ruff, and mypy are clean. I?m running the full coverage suite as the last local merge gate.

386. assistant (2026-07-07T13:39:22.710Z)

The verified local gate passed: 430 tests, 96.16% coverage, plus Ruff/mypy clean. Since the reviewer explicitly said merge-ready, I?m merging PR #48 now; optional follow-ups will ride later tracker slices.

387. assistant (2026-07-07T13:39:35.074Z)

The local cleanup hit the shared-worktree guard again because `main` is checked out in the primary worktree. I?m checking the remote PR state; in the previous PRs this happened after the remote merge completed.

388. assistant (2026-07-07T13:39:45.988Z)

PR #48 did merge remotely at `1433ee6`; only the remote topic branch was left behind. I?m deleting that branch, then I?ll refresh from `origin/main` before starting K4.

389. assistant (2026-07-07T13:39:56.092Z)

K3 is merged and its remote branch is cleaned up. I?m creating a fresh K4 worktree from `origin/main` now, so the lint skeleton starts from the merged tracker state.

390. assistant (2026-07-07T13:40:11.738Z)

The K4 worktree is created from `origin/main` at the #48 merge commit. I?m doing the required pull check inside it and then reading the lint/schema/test neighborhood.

391. assistant (2026-07-07T13:40:23.314Z)

K4 is clean and up to date. I?m reading the current CLI, schema, and baseline eval code to decide where the read-only lint skeleton should fit with the least churn.

392. assistant (2026-07-07T13:40:35.357Z)

K4?s shape is clearer now: a read-only `baseline lint` command with deterministic findings. I?m going to keep it as a skeleton with concrete first checks and warning-level placeholders for deeper stale/contradiction work, so it can land safely before K6 writes project feeds.

393. assistant (2026-07-07T13:40:51.161Z)

Current project pages are mostly scaffold stubs, so K4 should not fail the repo just because K6 has not populated cross-links yet. I?m going to make orphan/stale/contradiction checks warning-level in the first skeleton, while malformed markers and broken links are errors.

394. assistant (2026-07-07T13:41:19.159Z)

I?m about to add a new `baseline_lint.py`, wire `baseline lint` into the CLI, and add tests for

the seeded K4 cases: malformed markers, broken generated links, stale blocks, orphan project pages, and explicit contradiction markers.

395. assistant (2026-07-07T13:42:14.273Z)

The lint module is in place with read-only checks and report rendering. I?m wiring it into `baseline lint` next.

396. assistant (2026-07-07T13:42:25.081Z)

`baseline lint` is wired into the CLI. I?m adding tests now, including the direct lint fixtures and parser/dispatch coverage.

397. assistant (2026-07-07T13:43:10.140Z)

One subtle bit in the tests: the shared fixture has configured pilots but does not scaffold their pages, which is useful here because orphan warnings stay explicit in the tests that create pages. I?m adding CLI/parser and import-hygiene coverage.

398. assistant (2026-07-07T13:43:23.255Z)

The K4 tests are wired. I?m running the new lint tests and CLI/import slices first, before doing docs.

399. assistant (2026-07-07T13:43:36.522Z)

Targeted tests pass and Ruff is clean. I?m running `baseline lint --dry-run` against the real repo snapshot now, to see the first report shape and catch any accidental existing errors.

400. assistant (2026-07-07T13:43:49.582Z)

The real repo lint report has zero errors and six expected orphan-page warnings, which is exactly the safe K4 boundary before K6 starts writing project feeds. I?m updating the schema/docs/tracker/handoff to document that behavior.

401. assistant (2026-07-07T13:43:56.018Z)

I?m patching the schema template and tracker together so the committed `baseline/SCHEMA.md` still matches `baseline_schema()`. Then I?ll refresh the handoff to say K3 is merged and K4 is active.

402. assistant (2026-07-07T13:44:11.169Z)

Schema docs now describe the K4 severity model. I?m updating the tracker rows and changelog next.

403. assistant (2026-07-07T13:44:19.831Z)

Tracker now has K3 done and K4 active. I?m updating the engineering overview and handoff to match the new state and to capture the PR #48 optional follow-ups.

404. assistant (2026-07-07T13:44:27.015Z)

The engineering doc now tells future contributors when to run the lint gate. I?m refreshing the handoff with K4?s current worktree and the non-blocking follow-ups from the K3 review.

405. assistant (2026-07-07T13:44:51.481Z)

I?m also going to carry the small non-blocking K3 review follow-up that fits this PR: centralize the project-page scaffold placeholder and add the golden empty-file upsert test. It protects the lint/upsert contract without changing K4?s read-only command behavior.

406. assistant (2026-07-07T13:44:59.436Z)

The constant names were a hair different than I guessed, so I?m checking the exact top of the settings module and then applying the small refactor precisely.

407. assistant (2026-07-07T13:45:24.465Z)

The placeholder is centralized now. I?m adding the two tests the reviewer suggested: actual `project_readme()` upsert and exact empty-file promoted output.

408. assistant (2026-07-07T13:45:57.658Z)

The review follow-up tests are in. I?m rerunning the affected baseline/lint tests now.

409. assistant (2026-07-07T13:46:13.726Z)

Affected tests are green: 150 passed, and Ruff is clean. I?m running mypy and the full coverage suite now so the docs can reflect real counts.

410. assistant (2026-07-07T13:46:40.052Z)

Full suite is green, but coverage is 95.89%, below the 96.16% base from K3. I?m tightening the lint tests now so this PR does not regress coverage.

411. assistant (2026-07-07T13:46:55.008Z)

The uncovered lint branches are all sensible edge cases, so I?m adding a compact batch: nested/unopened/unclosed markers, skipped external/anchor links, missing baseline root, no-finding report, and relative output path.

412. assistant (2026-07-07T13:47:23.092Z)

Those edge tests should recover the coverage drop. I?m rerunning the focused lint suite, then full coverage if it?s clean.

413. assistant (2026-07-07T13:47:29.686Z)

Focused lint tests are clean. Running the full gate again to confirm coverage has recovered above the K3 baseline.

414. assistant (2026-07-07T13:47:58.405Z)

Coverage is now 96.40%, above the K3 base. I?m grabbing exact per-file test counts for the test plan, then I?ll patch the docs.

415. assistant (2026-07-07T13:49:20.481Z)

I?ll do a quick review-state sweep first, then continue from the tracker flow if there?s no new blocker. I?m going to respect the repo freshness rule and avoid touching the dirty primary checkout.

416. assistant (2026-07-07T13:49:30.215Z)

The primary checkout is still intentionally dirty on `SESSION_HANDOFF.md`, so the freshness pull refused rather than overwriting it. I?m switching back to the isolated worktree and will poll GitHub from there.

417. assistant (2026-07-07T13:49:41.692Z)

There are no open PRs right now, so the review that ?picked up? appears to have already landed and been merged. I?m continuing with the next tracker item in the isolated `codex/baseline-lint` worktree and tightening the docs before opening that PR.

418. assistant (2026-07-07T13:49:48.670Z)

The focused collection confirms the new lint tests add the expected surface: baseline now has 78 tests, baseline lint has 16, CLI has 47, and module-import coverage includes the new module. I?m folding those counts into the test plan now.

419. assistant (2026-07-07T13:49:54.932Z)

I?m going to make the test plan reflect the current repo rather than the old starting point: new lint test file, updated counts, and the latest full coverage gate.

420. assistant (2026-07-07T13:50:17.884Z)

The first patch targeted the default workspace path, so I?m switching to absolute filenames for this worktree. No file changed from that failed patch.

421. assistant (2026-07-07T13:50:25.653Z)

Minor PowerShell syntax stumble; I?ll reread the relevant slices with the range expression wrapped correctly.

422. assistant (2026-07-07T13:50:28.844Z)

I?ve got the exact context now; applying the doc update directly to the K4 worktree path.

423. assistant (2026-07-07T13:50:51.066Z)

The test plan is updated. I?m going to do one last PR-state check around the just-merged review, then run the K4 quality gate from this worktree.

424. assistant (2026-07-07T13:51:00.067Z)

PR #48 is confirmed merged at `2026-07-07T13:39:26Z`, and there are no open PRs. Ruff and whitespace diff checks are clean for K4; I?m running mypy, the full coverage suite, and the new lint command against the repo next.

425. assistant (2026-07-07T13:51:29.283Z)

The full gate is green: mypy clean and 450 tests passed at 96.40% coverage. One command form didn?t print because `agent_sessions.cli` isn?t an executable module entrypoint, so I?m invoking the CLI `main()` directly to capture the lint report.

426. assistant (2026-07-07T13:51:46.958Z)

The lint report is intentionally warning-only on today?s scaffold pages: 0 errors, 6 orphan-page warnings that K6?s generated feeds should retire. I?m doing the branch-safety checks and a quick self-review pass before staging explicit paths.

427. assistant (2026-07-07T13:51:58.918Z)

Self-review found one useful missing edge case: an end marker with no matching begin marker. It?s small and cheap, so I?m adding that test instead of leaving the branch uncovered.

428. assistant (2026-07-07T13:52:17.205Z)

That edge test is in, and I adjusted the expected suite count. I?m rerunning the full gate so the PR body and docs use verified numbers, not guesses.

429. assistant (2026-07-07T13:52:36.419Z)

The new edge test brought `baseline_lint.py` to 100% coverage and the full suite to 451 passed at 96.48%. I?m correcting the test plan to that exact verified number.

430. assistant (2026-07-07T13:52:48.821Z)

Docs now match the verified gate. I?m doing the required pre-stage branch check, then I?ll stage only the files for K4 explicitly.

431. assistant (2026-07-07T13:52:54.590Z)

The branch is still sitting directly on `origin/main` with no extra commits, so staging is cleanly scoped. I?m staging the K4 files now and committing them as one PR-ready change.

432. assistant (2026-07-07T13:53:04.656Z)

K4 is committed locally. I?m reading the PR-publish workflow note, then I?ll verify only this commit is ahead, push, and open the PR.

433. assistant (2026-07-07T13:53:14.027Z)

The branch is ahead by exactly one K4 commit. I?m using `gh` for the private repo path here, pushing the branch and opening a ready review PR so the reviewer can pick it up.

434. assistant (2026-07-07T13:53:18.097Z)

Authentication and remote metadata look good: repo is `avidullu/agent-sessions`, base `main`. Pushing the branch now.

435. assistant (2026-07-07T13:53:32.096Z)

The branch is on GitHub. I?m opening the K4 PR with the validation details and the known lint warnings called out directly in the body.

436. assistant (2026-07-07T13:53:49.928Z)

PR #49 is open. I?m updating the tracker and handoff with the actual PR number, then I?ll push that docs-only follow-up commit to the same branch.

437. assistant (2026-07-07T13:54:05.151Z)

CI has started on #49. While updating the PR number, I noticed one K3 follow-up is better to finish now: documenting that placeholder removal is exact-line only, which is the safer behavior the reviewer called out.

438. assistant (2026-07-07T13:54:12.189Z)

The current implementation already drops only an exact placeholder line. I?m adding a regression test for an edited placeholder-like sentence and a short schema/doc note so this behavior is explicit.

439. assistant (2026-07-07T13:54:26.880Z)

Now I?m adding the matching docs note in both the committed schema and the schema template so scaffolded repos inherit the same contract.

440. assistant (2026-07-07T13:54:32.090Z)

The live schema has the exact-line note; I?m mirroring it into the generated schema template and the engineering guide.

441. assistant (2026-07-07T13:54:37.329Z)

One more documentation touch: the engineering guide gets the operational version of that same rule.

442. assistant (2026-07-07T13:54:47.630Z)

That finishes the optional K3 documentation/test follow-up inside K4. I?m rerunning the focused

and full gates, then I?ll update the test-plan counts from the verified result.

443. assistant (2026-07-07T13:55:07.700Z)

The re-run is green: 452 passed, 96.48%. I?m updating the tracker/handoff and test-plan count before the docs follow-up commit.

444. assistant (2026-07-07T13:55:17.036Z)

The test plan is current. I?m putting PR #49 into the project tracker and handoff so the next resume lands in the right place.

445. assistant (2026-07-07T13:55:38.411Z)

Handoff now points to PR #49 and notes which K3 review follow-ups were folded in. I?m doing a final diff/branch check for the follow-up commit.

446. assistant (2026-07-07T13:55:48.069Z)

The follow-up diff is scoped and branch history still contains only the K4 commit. I?m making one tiny changelog-order cleanup, then committing and pushing this docs/test follow-up.

447. assistant (2026-07-07T13:55:53.844Z)

Changelog order is cleaned up. I?m staging only the follow-up files and pushing the second commit to #49.

448. assistant (2026-07-07T13:56:03.837Z)

The follow-up commit is pushed to #49. I?m updating the PR body so the validation line says 452 tests and calls out the exact-placeholder behavior.

449. assistant (2026-07-07T13:56:27.591Z)

PR #49?s body is up to date. I?m starting the requested polling loop now: first check covers CI status, review decision, and thread-aware comments.

450. assistant (2026-07-07T13:56:53.129Z)

The poll found the review: reviewer posted LGTM and explicitly said merge-ready. They listed three optional/non-blocking follow-ups; the only fresh one is invalid-date handling, and they also said those can ride K6/K12, so I?m waiting for CI on the latest pushed commit before merging.

451. assistant (2026-07-07T13:57:02.895Z)

CI is green and merge state is clean. I?m merging #49 now, then I?ll verify the PR state and remote branch cleanup.

452. assistant (2026-07-07T13:57:15.259Z)

`gh pr merge` hit the usual local-worktree checkout limitation because `main` is checked out in the primary workspace. I?m checking whether the remote merge still completed, then I?ll clean up the remote branch if needed.

453. assistant (2026-07-07T13:57:35.011Z)

#49 is merged remotely at `2026-07-07T13:57:06Z`; the local command failed only during its checkout cleanup step. The remote branch still exists, so I?m deleting it manually and then I?ll advance to the next tracker row.

454. assistant (2026-07-07T13:57:51.637Z)

I?m creating a fresh K5 worktree from `origin/main` so the next PR starts from the merge commit and doesn?t inherit any local checkout state.

455. assistant (2026-07-07T13:58:10.418Z)

Fresh K5 branch is on the #49 merge commit. I?m pulling fast-forward in the new worktree and reading the tracker/schema/ingest code to scope the smallest K5 slice.

456. assistant (2026-07-07T13:58:31.396Z)

K5 scope is the provenance-integrity slice: thread structured trace fields through `Prediction`/proposal ingest and reject unresolved `session_id` or `markdown_path` references against `archive/index.jsonl`. I?m reading the existing ingest/types tests before editing.

457. assistant (2026-07-07T13:58:40.544Z)

The current proposal schema already advertises `trace`, but ingest ignores it. K5 will make that field first-class and validate archive references only when structured trace/replay fields are supplied, preserving hand-written free-text proposals.

458. assistant (2026-07-07T13:58:51.926Z)

The archive index stores markdown paths with Windows separators and session ids under `metadata.session_id`. I?m checking the K5 design section now so validation semantics match the tracker rather than inventing a stricter shape.

459. assistant (2026-07-07T13:59:04.477Z)

I?m going to keep compatibility low-risk: add a `trace` field to `Prediction` with a default empty tuple, serialize it only when present, and validate structured trace only for `source_kind` of `replay` or `handoff`.

460. assistant (2026-07-07T13:59:55.243Z)

Before editing, the important compatibility choice: `source_kind`/`trace` are optional for existing human proposals, but replay/handoff proposals now need structured trace and resolvable archive references. That keeps today?s ingest usable while blocking untraceable external producers.

461. assistant (2026-07-07T14:00:02.669Z)

`Prediction` now has an optional trace tuple and only serializes trace when non-empty, so existing sidecars don?t grow noisy empty fields. Next I?m adding archive-reference validation to proposal ingest.

462. assistant (2026-07-07T14:00:46.203Z)

The ingest code now has the reference model. I?m adding tests for the three important paths: human proposals still pass without an index, external proposals without trace are rejected, and valid trace references survive into the sidecar.

463. assistant (2026-07-07T14:01:21.567Z)

The tests now pin reference rejection and sidecar propagation. I?m running the focused ingest/type tests and mypy next; if the types complain, I?ll tighten the helper annotations.

464. assistant (2026-07-07T14:01:34.232Z)

Focused tests and type checks are clean. I?m updating the schema/template/proposal schema and tracker docs so the new source-kind and trace validation contract is documented with the code.

465. assistant (2026-07-07T14:01:43.726Z)

I noticed the tracker?s JSON example uses `source_kind: "repo-handoff"` while the prose says `handoff`. I?m accepting that alias too so the shipped example won?t be rejected by K5 validation.

466. assistant (2026-07-07T14:01:53.132Z)

The alias is covered by tests now. I?m adding `source_kind` and `replay_of` to both the runtime proposal schema and the checked-in example schema.

467. assistant (2026-07-07T14:02:05.082Z)

Now the docs: I?m making K5?s validation contract explicit in the schema and the scaffold template.

468. assistant (2026-07-07T14:02:15.069Z)

Committed schema updated; mirroring the same contract into `baseline_schema()` so future scaffolds stay in sync.

469. assistant (2026-07-07T14:02:31.720Z)

The schema contract is updated. I?m updating the tracker state and closing the open K5 design question now that trace fields live directly on proposal objects and are copied into `Prediction.trace`.

470. assistant (2026-07-07T14:02:35.521Z)

The tracker still shows #49 as in progress from before the merge; I?m marking that done and setting K5 as the active row.

471. assistant (2026-07-07T14:02:49.884Z)

The design questions are now answered in the tracker. I?m tightening the proposal-schema test to assert the new fields too.

472. assistant (2026-07-07T14:02:56.143Z)

I?m refreshing the handoff so a restart sees #49 as merged and K5 as the current branch, plus the non-blocking lint follow-ups from the #49 review.

473. assistant (2026-07-07T14:03:16.064Z)

Handoff is now honest about the shipped state and K5?s current validation decision. I?m doing a focused test run after the docs/schema changes before updating the overall test-plan counts.

474. assistant (2026-07-07T14:03:25.407Z)

Focused checks are clean. I?m running the full coverage gate now; then I?ll update the test plan with the verified suite total and the `test_baseline_ingest.py` count.

475. assistant (2026-07-07T14:03:50.245Z)

Full suite is still 96.48%, now 459 tests. I?m updating `docs/TEST_PLAN.md` to add the ingest coverage row and the verified count.

476. assistant (2026-07-07T14:04:10.593Z)

One small cleanup before final checks: I?m making ?archive index required? depend on actual structured trace objects, so a malformed trace reports the shape problem without also pretending references were validated.

477. assistant (2026-07-07T14:04:23.443Z)

The validation helper is a touch cleaner now. I?m running focused checks again and then the full PR gate one more time after the test-plan doc change.

478. assistant (2026-07-07T14:04:28.293Z)

Focused ingest, ruff, mypy, and whitespace checks are clean. I?m running the full coverage suite again as the final local gate before committing.

479. assistant (2026-07-07T14:04:48.741Z)

Final full gate improved to 96.52% with 459 tests. I?m correcting the test plan coverage number.

480. assistant (2026-07-07T14:05:02.696Z)

The test plan now matches the verified gate. I?m doing the final self-review diff and branch-safety checks before staging.

481. assistant (2026-07-07T14:05:08.936Z)

Self-review looks good; one doc ordering nit remains in the tracker changelog, then I?ll stage and commit the K5 slice.

482. assistant (2026-07-07T14:05:18.597Z)

The branch is still cleanly based on `origin/main`; I?m doing the final branch-safety check and staging only the K5 files.

483. assistant (2026-07-07T14:05:25.128Z)

Branch-safety checks are clean, with no commits ahead yet. I?m committing the K5 trace-reference validation slice now.

484. assistant (2026-07-07T14:05:35.233Z)

K5 is committed locally. I?m verifying the branch contains only that commit, then pushing and opening the PR with the full gate results.

485. assistant (2026-07-07T14:05:41.723Z)

The branch is ahead by exactly one K5 commit. Pushing and opening the ready-for-review PR now.

486. assistant (2026-07-07T14:05:48.789Z)

Branch pushed. I?m creating the K5 PR with the validation details and scope notes.

487. assistant (2026-07-07T14:06:04.823Z)

PR #50 is open. I?m updating the tracker/handoff with the actual PR number, then I?ll push that tiny docs follow-up to the same branch.

488. assistant (2026-07-07T14:06:09.152Z)

CI is queued for #50. I?m adding the PR number into the tracker and handoff now.

489. assistant (2026-07-07T14:06:15.993Z)

Tracker has #50. I?m adding the same pointer to the handoff.

490. assistant (2026-07-07T14:06:22.872Z)

Only tracker and handoff changed after PR creation. I?m committing that docs-only PR-number update and pushing it to #50.

491. assistant (2026-07-07T14:06:36.400Z)

Docs follow-up is pushed. I?m starting the #50 polling loop now: CI, review state, and thread-aware comments.

492. assistant (2026-07-07T14:06:43.619Z)

First #50 poll: CI is in progress, no comments or review threads yet. I?m waiting on the check, then I?ll poll review state again.

493. assistant (2026-07-07T14:07:24.205Z)

CI is green on #50. I?m polling the review/comment state again now.

494. assistant (2026-07-07T14:07:34.702Z)

Second #50 poll: CI passed, merge state clean, no reviews/comments yet. I?ll do a few more polls on the 10-minute cadence window before deciding whether to pause on review.

495. assistant (2026-07-07T14:08:11.462Z)

Third #50 poll: still clean and no review activity. I?m doing another thread-aware poll after a short wait so inline comments don?t get missed.

496. assistant (2026-07-07T14:08:50.545Z)

Fourth #50 poll: no reviews or threads yet. I?ll do one more short poll, then I?ll leave it waiting for reviewer input unless something appears.

497. assistant (2026-07-07T14:09:36.572Z)

Fifth #50 poll: still no review or comments; CI remains green and the PR is merge-clean. I?m checking local status now and then I?ll pause on reviewer input for #50.

498. assistant (2026-07-07T14:10:03.296Z)

Polled and acted on the review flow.

PR #49 got reviewer LGTM / merge-ready, CI was green, and I merged it. Remote branch cleanup is done.

I then moved to the next tracker item and opened [PR #50](https://github.com/avidullu/agent-sessions/pull/50) for K5 trace-reference validation. It is open, CI is green, merge state is clean, and after several polls there are no review comments or threads yet.

Local gate for #50:
`ruff check .` clean
`mypy agent_sessions tools` clean
`pytest --cov=agent_sessions --cov-report=term-missing`: 459 passed, 96.52%

Pausing on #50 review input now.

::git-stage{cwd="C:/Users/avidu/Projects/Agent Sessions - baseline-lint"}
::git-commit{cwd="C:/Users/avidu/Projects/Agent Sessions - baseline-lint"}
::git-push{cwd="C:/Users/avidu/Projects/Agent Sessions - baseline-lint" branch="codex/baseline-lint"}
::git-create-pr{cwd="C:/Users/avidu/Projects/Agent Sessions - baseline-lint" branch="codex/baseline-lint" url="https://github.com/avidullu/agent-sessions/pull/49" isDraft=false}
::git-create-branch{cwd="C:/Users/avidu/Projects/Agent Sessions - trace-reference-validation" branch="codex/trace-reference-validation"}
::git-stage{cwd="C:/Users/avidu/Projects/Agent Sessions - trace-reference-validation"}
::git-commit{cwd="C:/Users/avidu/Projects/Agent Sessions - trace-reference-validation"}
::git-push{cwd="C:/Users/avidu/Projects/Agent Sessions - trace-reference-validation"

branch="codex/trace-reference-validation"}
::git-create-pr{cwd="C:/Users/avidu/Projects/Agent Sessions - trace-reference-validation"
branch="codex/trace-reference-validation" url="https://github.com/avidullu/agent-
sessions/pull/50" isDraft=false}

499. user (2026-07-07T14:19:13.084Z)

PR has review comments. pls proceed

500. assistant (2026-07-07T14:19:30.159Z)

I?ll pick this up on PR #50, refresh the branch first, then read the thread-aware review state so inline comments don?t get flattened away.

501. assistant (2026-07-07T14:19:42.395Z)

Using the GitHub review-comments workflow here; the branch is up to date. I?m pulling the review threads and top-level review bodies now.

502. assistant (2026-07-07T14:20:01.968Z)

The review is an LGTM and explicitly says merge-ready. There?s one optional note about `lstrip("./")`; since it?s non-blocking and the reviewer cleared merge, I?m going to re-run the merge gate on the exact PR head, merge #50, then move to the next tracker item.

503. assistant (2026-07-07T14:20:07.122Z)

Ruff, mypy, whitespace, and GitHub CI are clean. I?m running the full coverage suite one more time before merging.

504. assistant (2026-07-07T14:20:33.292Z)

The final merge gate is green: 459 passed at 96.52%. I?m verifying branch contents one last time and then merging #50.

505. assistant (2026-07-07T14:20:53.907Z)

The `gh pr merge` command hit the local worktree checkout limitation again, so I?m checking the remote state before doing anything else.

506. assistant (2026-07-07T14:21:04.253Z)

#50 is merged remotely at `2026-07-07T14:20:44Z`; only the local checkout cleanup failed. I?m deleting the remote feature branch manually, then I?ll cut a fresh K6 branch from `origin/main`.

507. assistant (2026-07-07T14:21:10.592Z)

Remote branch cleanup is done. Next tracker item is K6: persistent handoff discovery records plus a project-page feed, based on the now-merged K5.

508. assistant (2026-07-07T14:21:19.114Z)

I?m creating the K6 worktree from the remote merge commit explicitly, keeping it isolated from the primary checkout.

509. assistant (2026-07-07T14:21:24.186Z)

No existing K6 worktree at that path, so I?m adding one from `origin/main`.

510. assistant (2026-07-07T14:21:43.417Z)

K6 branch is ready on the #50 merge commit. I?m refreshing it and reading the handoff/audit/upsert code to keep this slice aligned with the existing K2/K3/K5 contracts.

511. assistant (2026-07-07T14:22:05.060Z)

The existing K2 audit already detects the candidates; K6 can reuse that scanner and add deterministic writes: `baseline/handoffs/index.jsonl` plus marker-owned project-page sections. I?m checking the CLI and project-page helpers before editing.

512. assistant (2026-07-07T14:22:30.275Z)

I?m choosing a conservative K6 shape: a new `baseline handoffs index` command that writes a deterministic JSONL index and upserts a single marker-owned `handoffs.index` block into each affected project page. No proposals yet; K7 still owns proposal generation.

513. assistant (2026-07-07T14:23:02.921Z)

Before editing, I?m going to add the command surface and pure helpers first, then tests, then run the real command once the behavior is pinned.

514. assistant (2026-07-07T14:23:59.559Z)

The core K6 helpers are in place. I?m wiring the new `baseline handoffs index` subcommand next.

515. assistant (2026-07-07T14:24:20.352Z)

I?m adding tests at the helper and command levels: slug derivation/collisions, index JSONL, project-page marker upsert, and dry-run write boundaries.

516. assistant (2026-07-07T14:24:56.151Z)

The handoff index tests are in. I?m adding the CLI parser/dispatch assertions so the new command surface is covered too.

517. assistant (2026-07-07T14:24:59.990Z)

The existing CLI tests have adjacent handoffs coverage, so I?m extending those spots with `handoffs index`.

518. assistant (2026-07-07T14:25:17.082Z)

The CLI test block had slightly different spacing than expected. I?m reading a wider slice and patching against the exact lines.

519. assistant (2026-07-07T14:25:22.506Z)

Now I have the exact blocks; applying the CLI coverage.

520. assistant (2026-07-07T14:25:45.158Z)

CLI coverage is in. I?m running the targeted K6 tests now; expect a couple of small type or formatting fixes after the first pass.

521. assistant (2026-07-07T14:25:54.930Z)

K6 targeted tests, ruff, and mypy are clean. I?m going to run the new command on the real archive now so the persistent index and project feeds are produced as part of the K6 PR.

522. assistant (2026-07-07T14:26:25.198Z)

The first real run surfaced useful data: several archived records carry noisy project path forms that produced ugly slugs. I?m going to tighten slug derivation before keeping generated outputs, then rerun so the committed feed is cleaner.

523. assistant (2026-07-07T14:26:45.260Z)

Those noisy slugs come from Claude?s hyphen-encoded project path format (`C--Users-...-Projects-...`). I?m teaching the slugger that format and pilot-alias substring matching, then I?ll remove the first bad generated pages and rerun the index.

524. assistant (2026-07-07T14:28:25.889Z)

I?ll pick this up from the repo state first, then I?ll inspect the active PR review threads and address anything actionable. I?ll keep the polling cadence long once we?re back to waiting.

525. assistant (2026-07-07T14:28:38.252Z)

The primary checkout can?t fast-forward because `SESSION_HANDOFF.md` has local changes, so I?m treating that checkout as read-only/stale and moving in the dedicated worktree. I also don?t see an open PR from the primary branch, so I?m checking the active worktree/remote PR list next.

526. assistant (2026-07-07T14:28:54.964Z)

The K6 worktree has local WIP but no open PR exists on GitHub right now. I?m going to verify whether the reviewer commented on a recently merged PR, then I?ll either respond there if needed or finish getting the current K6 branch into a PR.

527. assistant (2026-07-07T14:29:04.817Z)

The latest PRs are already merged, and there are currently no open PRs. I?m still checking PR #50?s review threads so I don?t miss a post-merge comment or an optional follow-up the reviewer meant us to carry forward.

528. assistant (2026-07-07T14:29:10.888Z)

The helper script defaults to the current branch PR, so it ignored the positional PR number I tried. I?m checking its exact flags and then I?ll fetch the relevant merged PR threads explicitly.

529. assistant (2026-07-07T14:29:23.530Z)

The comment helper only supports ?current branch PR?, so I?ll use the existing PR #50 worktree for the thread-aware read. If that turns out clean, the next real work is finishing the current K6 branch and opening its PR.

530. assistant (2026-07-07T14:29:39.890Z)

I found the ?comments? that still exist: PR #49 and #50 both have LGTM reviews with optional follow-ups, not blocking review threads. I?ll fold the small concrete ones into the current K6 work where they fit, and keep anything broader parked for the tracker instead of smuggling in a refactor.

531. assistant (2026-07-07T14:29:50.706Z)

I?m checking the K6 diff now, especially the slugging/feed behavior that created noisy project pages during the first dry-ish real run. The next edit will tighten that so the durable index keeps all records while project page feeds stay to known/scaffolded projects.

532. assistant (2026-07-07T14:29:58.576Z)

The slug fix did land, but the page-upsert path still writes pages for every raw slug. I?m going to constrain that: the JSONL index remains comprehensive, while project-page feeds only update configured or existing project pages.

533. assistant (2026-07-07T14:30:02.434Z)

I?ll make two focused edits: add a ?known project pages only? guard to the feed upsert, and add tests for the hyphen-encoded Windows paths that showed up in the real archive. After that I?ll clean the generated noise from the first run and regenerate.

534. assistant (2026-07-07T14:30:06.727Z)

I?m editing the feed writer now so K6 is durable but restrained: all handoffs go into `baseline/handoffs/index.jsonl`; only known project pages get marker blocks.

535. assistant (2026-07-07T14:30:16.009Z)

Next I?m tightening the tests around exactly what bit us: encoded local project paths and avoiding surprise page creation for unknown slugs.

536. assistant (2026-07-07T14:30:34.455Z)

I?m also folding in the two small review follow-ups that are low-risk: the path-prefix normalization nit from #50 and the invalid generated-date guard from #49. Broader taxonomy/perf notes will stay as tracker material.

537. assistant (2026-07-07T14:30:45.111Z)

The optional fixes are small and testable, so I?m adding regression tests with them rather than just changing the code.

538. assistant (2026-07-07T14:30:54.946Z)

I?m keeping the lint date fix as a warning, so a malformed marker date becomes visible without aborting an otherwise read-only lint run.

539. assistant (2026-07-07T14:31:00.908Z)

Now I?m applying the stale-date guard and its regression coverage.

540. assistant (2026-07-07T14:31:07.470Z)

The code fixes are in; I?m adding the tests now so the reviewer has concrete proof for both PR-comment follow-ups.

541. assistant (2026-07-07T14:31:14.032Z)

One more regression test covers the invalid date behavior, then I?ll run the focused suite before regenerating artifacts.

542. assistant (2026-07-07T14:31:29.836Z)

I?m running the focused tests and lint on the touched modules now. If that?s clean, I?ll clean up the earlier generated noise and rerun K6 for real.

543. assistant (2026-07-07T14:31:43.292Z)

The focused checks pass. I?m deleting only the untracked project directories and index file produced by the earlier noisy generation, with path checks scoped under this K6 worktree, then I?ll regenerate with the stricter feed rule.

544. assistant (2026-07-07T14:32:04.030Z)

The noisy untracked pages are gone. I?m regenerating the handoff index/feed now and then I?ll inspect the resulting slugs and modified pages.

545. assistant (2026-07-07T14:32:30.839Z)

Regeneration now writes only known project pages. I see one earlier modified page (`badminton-highlight-indexer`) that the current run did not touch, so I?m checking whether it should be reverted from this branch or if the index now legitimately includes it under a disambiguated slug.

546. assistant (2026-07-07T14:32:55.891Z)

I found the cause: the collision guard was overzealous for configured pilot aliases, so `badminton-highlight-indexer` got split into digest-suffixed slugs. I?m fixing that now and making alias matching choose the strongest project alias instead of accidentally matching a username in a full path.

547. assistant (2026-07-07T14:33:06.077Z)

I?m adding a regression for ?configured pilots do not get digest-split? and for longest alias wins on full paths that include the username.

548. assistant (2026-07-07T14:33:16.041Z)

The pilot collision behavior is now covered with two artificial raw paths that should both stay on the configured pilot slug.

549. assistant (2026-07-07T14:33:31.516Z)

I?m also adding a ?longest alias wins? test to prevent user/home path fragments from beating a real project/org alias.

550. assistant (2026-07-07T14:34:00.109Z)

I?m re-running the focused checks after the slug changes, then I?ll regenerate the index once more from a clean generated-output state.

551. assistant (2026-07-07T14:34:05.927Z)

The slug tests pass. I?m regenerating again now; afterward I?ll verify that configured pilots are no longer split into digest-suffixed slugs and clean any stale block left from the earlier run.

552. assistant (2026-07-07T14:34:27.630Z)

The regenerated index looks saner: configured pilots are intact (`badminton-highlight-indexer` has 62, `khelsutra` has 11). The only leftover is the stale `avidullu` block from the earlier noisy run, so I?m removing that local artifact.

553. assistant (2026-07-07T14:34:32.480Z)

I?m patching `avidullu` back to its scaffold text since current generation no longer writes that page.

554. assistant (2026-07-07T14:34:49.325Z)

I?ll verify the generated artifact set now: no stray pages, the expected project feeds only, and the status should no longer include `avidullu`.

555. assistant (2026-07-07T14:35:00.121Z)

The generated set is now tight: one JSONL index plus four known project feed pages, with no digest-suffixed pilot slugs. I?m moving to docs/tracker/handoff updates so the branch records the K6 scope and the reviewer follow-up decisions clearly.

556. assistant (2026-07-07T14:35:21.385Z)

I?m updating the plan and schema docs to reflect the implemented K6 behavior: persistent JSONL for all handoff candidates, marker feed blocks only on configured/existing project pages, and the small #49/#50 review follow-ups.

557. assistant (2026-07-07T14:35:34.305Z)

I?m mirroring the schema doc updates into the generated scaffold template as well, so `baseline scaffold` won?t drift from `baseline/SCHEMA.md`.

558. assistant (2026-07-07T14:35:49.162Z)

The checked-in schema is updated; I?m applying the same wording to `baseline_settings.baseline_schema()` to keep the scaffold golden test meaningful.

559. assistant (2026-07-07T14:35:59.666Z)

I?m updating the architecture doc next so it no longer describes persistent handoff feeds as ?later gated work.?

560. assistant (2026-07-07T14:36:09.975Z)

I?m refreshing the session handoff now, since K5 merged and K6 has real generated artifacts plus a couple of review follow-ups folded in.

561. assistant (2026-07-07T14:36:30.111Z)

I?m doing a quick docs search for stale references before running the bigger gate, especially anything that still says K6 is ?future? or that handoff indexing writes proposals.

562. assistant (2026-07-07T14:36:40.156Z)

The search caught one stale phrase in the K2 audit text that still says K6 owns proposal writes. I?m correcting that split so K6 is index/feed and K7 is proposal generation.

563. assistant (2026-07-07T14:36:43.612Z)

I?m patching the report wording and the existing audit artifact so they agree with the K6/K7 boundary.

564. assistant (2026-07-07T14:36:49.435Z)

The generator wording is fixed; I?m making the checked-in audit report match it.

565. assistant (2026-07-07T14:37:00.302Z)

I?ll run the focused checks once more because that wording is covered by the audit tests.

566. assistant (2026-07-07T14:37:17.128Z)

One thing I want to tighten before the full gate: the project-page links display repo-relative archive paths, but the Markdown target should be relative from `baseline/projects/<slug>/README.md` so GitHub actually resolves them. I?m fixing the rendered link target and regenerating the feeds.

567. assistant (2026-07-07T14:37:23.520Z)

I?m adding a small assertion for the project-page link target so this doesn?t drift back.

568. assistant (2026-07-07T14:37:30.097Z)

I?m regenerating the K6 outputs once more with the corrected relative Markdown links.

569. assistant (2026-07-07T14:37:57.321Z)

The project feed links now point back to the repo root correctly. I?ll run the focused tests plus `baseline lint --dry-run` before the full gate.

570. assistant (2026-07-07T14:38:05.500Z)

`baseline lint` is clean on errors but still warns on orphan project pages. Since K4?s review expected K6 to retire at least the generated-page warnings, I?m checking whether to add an index link from the baseline root or suppress pages with generated blocks.

571. assistant (2026-07-07T14:38:16.985Z)

K4?s orphan warning already checks whether a project page has generated blocks, but it still warned even when K6 added a feed. I?m tightening that rule so K6-fed pages stop warning; empty/scaffold pages still surface as orphan review signals.

572. assistant (2026-07-07T14:38:24.025Z)

I?m adding the lint regression so pages with K6 marker blocks are no longer reported as empty orphan pages.

573. assistant (2026-07-07T14:38:34.778Z)

I?m updating the schema wording for the orphan rule so docs match the refined lint behavior.

574. assistant (2026-07-07T14:38:52.783Z)

I?ll rerun the focused tests and `baseline lint`; this should leave only the genuinely empty/scaffold pages as orphan warnings.

575. assistant (2026-07-07T14:39:03.091Z)

The focused gate is clean and `baseline lint` is down to two expected scaffold warnings (`agent-sessions`, `avidullu`). I?m starting the full merge gate now: whitespace diff check, repo-wide ruff, mypy, and then the full coverage run.

576. assistant (2026-07-07T14:39:10.298Z)

Whitespace, ruff, and mypy are clean. I?m running the full pytest suite with coverage now; this is the slower but decisive gate.

577. assistant (2026-07-07T14:39:36.781Z)

The suite is green, but coverage is 96.21%, below the previous 96.52% floor from K5. I?m going to add a few focused tests around the new K6 branches instead of accepting the regression.

578. assistant (2026-07-07T14:39:52.078Z)

Most uncovered lines are branches in the new handoff index and renderer. I?m adding compact tests for relative outputs, index loading, unknown/non-colliding slugs, ?more than 10? feed rendering, and a few parser edge cases.

579. assistant (2026-07-07T14:40:07.360Z)

I?ll keep the coverage patch tight: no new behavior, just exercising branches already present in K6.

580. assistant (2026-07-07T14:40:19.813Z)

I added the helper and one parser branch; now I?m adding the index/load/render edge cases.

581. assistant (2026-07-07T14:40:46.850Z)

I spotted a typo in that coverage test before running it: it should call the existing `handoff_index_record()` helper. I?m correcting that now.

582. assistant (2026-07-07T14:40:52.577Z)

I?m adding one scan/parser edge test to cover malformed archive records and non-dict metadata

handling.

583. assistant (2026-07-07T14:41:06.180Z)

The new tests are in. I?m running the handoff test file and ruff to catch any style or coverage-test mistakes quickly.

584. assistant (2026-07-07T14:41:18.876Z)

The added coverage tests pass. I?m re-running the full gate now; this should bring coverage back above the K5 baseline.

585. assistant (2026-07-07T14:41:22.676Z)

Whitespace, ruff, and mypy are still clean. Running the full coverage suite again.

586. assistant (2026-07-07T14:41:47.147Z)

Full gate is green now: 480 passed with 96.90% coverage, above the K5 baseline. I?m updating the test plan counts to match the verified run.

587. assistant (2026-07-07T14:42:08.056Z)

I?m doing a final status and whitespace pass after the docs update, then I?ll stage explicit paths and commit the K6 branch.

588. assistant (2026-07-07T14:42:14.247Z)

The final local checks are good: `baseline lint` has only the two expected scaffold warnings. I?m doing the branch-safety check now before staging explicit paths.

589. assistant (2026-07-07T14:42:22.499Z)

HEAD is still exactly `origin/main`, so the branch is cleanly based. I?m staging only the K6 paths explicitly.

590. assistant (2026-07-07T14:42:30.281Z)

The staged set is exactly the K6 code, generated artifacts, docs, and tests. I?m committing it now.

591. assistant (2026-07-07T14:42:35.964Z)

Commit created. I?m verifying the branch contains only this commit over `origin/main`, then I?ll push and open the PR.

592. assistant (2026-07-07T14:42:48.065Z)

The branch is pushed with only the K6 commit. I?m opening the PR with the verification details and the folded #49/#50 follow-ups called out.

593. assistant (2026-07-07T14:43:06.315Z)

PR #51 is open. I?m recording the PR number in the tracker/handoff with a tiny follow-up commit, then I?ll push that and start polling for review.

594. assistant (2026-07-07T14:43:13.914Z)

The tracker has the PR number; I?m updating the session handoff to point future sessions at #51.

595. assistant (2026-07-07T14:43:19.361Z)

This is docs-only now. I?ll run a whitespace check, stage just the tracker/handoff, commit, and

push.

596. assistant (2026-07-07T14:43:24.200Z)

Whitespace is clean and only the two doc files changed. I?m committing the PR-number update.

597. assistant (2026-07-07T14:43:31.405Z)

The branch still contains only the two K6 commits over `origin/main`. I?m pushing the PR-number update now.

598. assistant (2026-07-07T14:43:41.767Z)

PR #51 is updated. I?m checking CI/review state now; if it?s just waiting, I?ll poll on the longer cadence you asked for.

599. assistant (2026-07-07T14:43:48.304Z)

PR #51 is open and CI is still pending (`test`). I?ll wait 10 minutes before the next poll, matching your requested cadence.

600. assistant (2026-07-07T14:54:07.275Z)

Review is in: LGTM, merge-ready, CI green on head `bccc200`. The notes are explicitly optional/non-blocking, so I?m merging PR #51 now and then I?ll move to the next tracker item.

601. assistant (2026-07-07T14:54:18.693Z)

`gh pr merge` hit the local worktree limitation again (`main` is checked out in the primary worktree). I?m verifying whether the remote merge completed; if it did, I?ll clean up the remote branch manually like the prior PRs.

602. assistant (2026-07-07T14:54:25.854Z)

PR #51 merged remotely at `2026-07-07T14:54:10Z` with merge commit `eb9f87f`. The remote branch is still present because the local cleanup step failed, so I?m deleting just the remote feature branch.

603. assistant (2026-07-07T14:54:33.706Z)

K6 is merged and its remote branch is deleted. I?m moving to the next tracker item, K7: handoff-derived proposal generation with structured trace records.

604. assistant (2026-07-07T14:54:42.136Z)

I?ll start K7 in a fresh worktree from `origin/main`, per the branch-safety convention, so it includes the just-merged K6 work and doesn?t inherit any local branch state.

605. assistant (2026-07-07T14:55:07.491Z)

The K7 worktree is clean and based on merged PR #51. I?m reading the proposal/ingest shape and current K6 index code so the next PR fits the existing contracts instead of inventing a new lane.

606. assistant (2026-07-07T14:55:29.768Z)

K7?s safest shape is deterministic proposal JSON, not synthesized policy. I?m going to make it generate review inputs from the K6 index for configured/existing projects only, with structured trace copied from the handoff records so `baseline ingest` can validate them before any candidate report is written.

607. assistant (2026-07-07T14:55:35.495Z)

I?ll inspect the CLI parser around the `baseline handoffs` subcommands and the existing proposal tests, then implement the new `baseline handoffs proposals` command in the same style.

608. assistant (2026-07-07T14:56:01.329Z)

A couple of guardrails I?m baking in: generated proposal files get a stable `generated_by` marker and the command refuses to overwrite a same-named non-generated proposal. No dates in the proposal payload, so repeated runs stay quiet when the index is unchanged.

609. assistant (2026-07-07T14:56:02.945Z)

I?m editing `baseline_handoffs.py` now to add the proposal builder/writer on top of the K6 index records.

610. assistant (2026-07-07T14:56:36.086Z)

The proposal builder is in. I?m wiring it into the CLI under `baseline handoffs proposals`.

611. assistant (2026-07-07T14:56:48.822Z)

I?m adding unit and command-level tests for K7: proposal shape, trace preservation/validation readiness, no unknown-page proposal generation, dry-run boundaries, and overwrite protection.

612. assistant (2026-07-07T14:56:54.083Z)

I?m patching the handoff tests with K7 helpers and command behavior now.

613. assistant (2026-07-07T14:57:13.536Z)

The K7 tests will also run the generated JSON through the existing proposal validator, so we know the trace references are shaped for K5 ingest.

614. assistant (2026-07-07T14:57:19.216Z)

I?m adding the proposal tests below the index tests so the progression stays audit -> index -> proposals.

615. assistant (2026-07-07T14:57:39.431Z)

Now I?m adding parser and dispatch coverage for the new CLI subcommand.

616. assistant (2026-07-07T14:58:00.441Z)

The K7 code and tests are patched. I?m running focused tests and ruff now; then I?ll generate the actual proposal JSON files from the merged K6 index.

617. assistant (2026-07-07T14:58:07.785Z)

Focused checks pass. I?m running `baseline handoffs proposals` for real now, using the K6 index on `origin/main`.

618. assistant (2026-07-07T14:58:15.995Z)

Four proposal files were generated for the configured projects that have handoff signals. I?m validating them through the existing ingest path before touching docs.

619. assistant (2026-07-07T14:58:32.117Z)

`baseline ingest --dry-run` accepts all five proposals (the existing example plus four generated handoff proposals), with zero rejections. I?m inspecting one generated proposal and then I?ll update the tracker/docs for K7.

620. assistant (2026-07-07T14:59:59.644Z)

I?ll pick this up through the GitHub review-comments workflow and first confirm which PR has actionable feedback, then I?ll patch the active worktree and re-run the local gate before replying on the PR.

621. assistant (2026-07-07T15:00:13.012Z)

The primary checkout can?t fast-forward because it has a local `SESSION_HANDOFF.md` edit, so I?m leaving that untouched and switching to the dedicated K7 worktree where the active PR work should live. Next I?m checking the open PR metadata and thread-level review state.

622. assistant (2026-07-07T15:00:21.959Z)

GitHub currently reports no open PRs for `avidullu/agent-sessions`, while the K7 worktree still has local uncommitted proposal-generation work and no PR open yet. I?m going to inspect the recently merged PRs for any unresolved/new comments so we don?t miss feedback that landed right after merge.

623. assistant (2026-07-07T15:00:30.950Z)

The recent PR list shows #51 is already merged and there are no open PRs, so I?m drilling into #51?s review threads/comments now. If the feedback is a post-merge note rather than a blocking change request, I?ll fold the fix into the next K7 PR.

624. assistant (2026-07-07T15:00:37.836Z)

The `gh pr view` surface doesn?t expose review-thread state here, so I?m using the bundled GraphQL helper the skill points to. That should tell me whether anything is unresolved, outdated, or just advisory.

625. assistant (2026-07-07T15:00:53.648Z)

The only current review feedback I can find is the post-merge #51 review body: one actionable code follow-up about project-page date churn, plus two documentation/future-work notes. I?ll fold those into the K7 branch now, alongside the proposal-generation work, so the next PR carries both the planned K7 slice and the reviewer follow-ups.

626. assistant (2026-07-07T15:01:03.354Z)

I found the concrete review item: `handoffs.index` blocks currently rewrite their generated date on every run. I?m going to make that date stable when the feed content is unchanged, then refresh the K7 docs/tracker/handoff before opening the next PR.

627. assistant (2026-07-07T15:01:11.987Z)

There?s also a small edge case in the K7 CLI: `--max-records-per-project 0` would create invalid proposal traces. I?m tightening that while I?m in the same area, since it?s exactly the kind of small thing a reviewer should not have to find for us.

628. assistant (2026-07-07T15:01:21.086Z)

The plan from here is concrete: adjust the feed renderer so it can reuse an existing block date if only the timestamp would change, add tests for that behavior and positive proposal limits, update the schema/tracker/handoff docs, then run the full lint/type/test/coverage gate before I push a PR.

629. assistant (2026-07-07T15:01:47.875Z)

I?m editing the handoff producer now: one helper to keep existing `Generated at` when the generated block is otherwise byte-equivalent, one guard for positive proposal citation limits, and a couple of focused tests around both.

630. assistant (2026-07-07T15:02:03.874Z)

The code change is in. I?m adding tests that lock the reviewer?s date-churn concern: unchanged feeds keep their prior date, changed feeds get the new date, and invalid citation limits fail cleanly.

631. assistant (2026-07-07T15:02:53.071Z)

The focused tests are in place. I?m running the handoff/CLI subset and ruff on the touched Python files before I move into docs, so any logic wobble gets caught early.

632. assistant (2026-07-07T15:03:08.934Z)

The focused suite is green: 33 passed, and ruff is clean on the touched Python/tests. I?m moving to the docs now, including the generated schema mirror so the schema golden check stays honest.

633. assistant (2026-07-07T15:03:26.133Z)

I?m updating the contract docs now: K7 proposal generation gets explicit ownership rules, the stable-date behavior is recorded, and the #51 prune/link notes are captured as future boundaries rather than silently dropping them.

634. assistant (2026-07-07T15:03:42.137Z)

The file-backed schema is updated; I?m mirroring the same text into the scaffold generator now so future `baseline scaffold` runs reproduce the contract exactly.

635. assistant (2026-07-07T15:04:03.965Z)

Next I?m refreshing the project tracker itself: #51 moves to done, K7 becomes the active row, and the reviewer?s optional notes are recorded as design constraints or #32 follow-ups.

636. assistant (2026-07-07T15:04:26.641Z)

I?m updating `SESSION_HANDOFF.md` as a real resumption note now, not as aspirational state: #51 is merged, K7 is active in this worktree, and the four generated proposal files are part of the ramp-up kit.

637. assistant (2026-07-07T15:04:47.336Z)

Docs are mostly refreshed. I?m regenerating the K7 proposal outputs and validating them through ingest now; this confirms the checked-in JSON still matches the command after the overwrite guard/stable-date edits.

638. assistant (2026-07-07T15:04:58.572Z)

The generated handoff proposals rewrote cleanly, and `baseline ingest --dry-run` accepts all 5 proposal files with 0 rejects. I?m checking the diff and then I?ll run the full local gate.

639. assistant (2026-07-07T15:05:15.760Z)

The diff looks scoped: handoff producer/CLI/tests, schema/tracker/handoff docs, and four generated proposal files. I?m running the full pre-PR gate now: whitespace diff check, ruff, mypy, baseline lint/ingest dry-runs, then the full coverage suite.

640. assistant (2026-07-07T15:05:22.022Z)

The static gate is clean: diff check, ruff, mypy, baseline lint, and proposal ingest all passed. `baseline lint` still reports the two known orphan warnings for `agent-sessions` and `avidullu`; no errors. I?m running the full coverage suite now.

641. assistant (2026-07-07T15:06:03.275Z)

The full suite passed, but coverage is 96.76%, which is above the repo floor but below the last recorded 96.90%. Since your gate explicitly asks for no coverage regression, I?m adding a few

focused edge-case tests for the new K7 branches rather than waving that through.

642. assistant (2026-07-07T15:06:14.541Z)

I found the uncovered new branches: relative `--index`/`--output-dir`, missing index errors, warning-derived proposal questions, invalid generated proposal JSON, and an existing block without a date. I?m adding those targeted tests; they should lift coverage back above the prior mark without broadening the implementation.

643. assistant (2026-07-07T15:06:50.795Z)

The additional tests are in. I?m re-running the focused handoff tests and ruff before the full suite, mostly to catch any awkward fixture interaction from the relative-path cases.

644. assistant (2026-07-07T15:06:56.886Z)

Focused handoff tests are now 36 passed, and ruff is clean. I?m running the full coverage suite again; if the percentage clears the prior 96.90%, I?ll update `docs/TEST_PLAN.md` with the new numbers.

645. assistant (2026-07-07T15:07:27.347Z)

Coverage is back above the prior mark: 495 passed with 97.05% total coverage. I?m updating the test-plan doc with those numbers, then I?ll rerun the shorter static gate after the doc edit.

646. assistant (2026-07-07T15:07:32.706Z)

The current counts are 51 CLI tests and 36 handoff tests. I?m patching the test-plan tracker so the documentation matches the suite I just ran.

647. assistant (2026-07-07T15:07:53.862Z)

The test-plan doc now reflects the 495/97.05 run. I?m doing one last local gate pass after the documentation edit, then I?ll verify branch ancestry and stage only the intended files.

648. assistant (2026-07-07T15:07:58.309Z)

Static checks are still clean, with the same two known lint warnings and proposal ingest accepting all 5 proposals. I?m re-running the full coverage suite once more now that the docs are final, then I?ll commit.

649. assistant (2026-07-07T15:08:34.681Z)

Full coverage is confirmed after the final doc edit: 495 passed, 97.05%. I?m doing the branch-safety checks now, then I?ll stage explicit paths and commit.

650. assistant (2026-07-07T15:08:41.315Z)

Branch base is exactly `origin/main` at #51?s merge commit, so no sibling commits are riding along. I?m staging only the intended files now.

651. assistant (2026-07-07T15:08:49.417Z)

The staged set is exactly the K7 code, docs, tests, and four generated proposal JSON files. I?m re-checking branch/HEAD immediately before commit, then I?ll commit and push.

652. assistant (2026-07-07T15:08:57.060Z)

Committed K7 locally. I?m verifying the PR range contains only that commit, then pushing the branch and opening the PR.

653. assistant (2026-07-07T15:09:01.633Z)

The PR range is clean: one commit, `d0b793c Add handoff-derived proposal generation`, on top of `origin/main`. Pushing now.

654. assistant (2026-07-07T15:09:11.619Z)

The branch is pushed. I?m opening the PR with the K7 scope and explicitly calling out that it addresses the #51 date-churn review note while leaving the prune semantics with #32.

655. assistant (2026-07-07T15:09:29.231Z)

PR #52 is open. I?m doing the usual follow-up doc/handoff commit now so the tracker points at the actual PR number instead of a placeholder.

656. assistant (2026-07-07T15:09:42.250Z)

The tracker and handoff now point to PR #52. I?m committing just those two doc files and pushing the update to the PR branch.

657. assistant (2026-07-07T15:09:45.696Z)

Whitespace check is clean, and only the PR-number docs are modified. Staging those explicit paths now.

658. assistant (2026-07-07T15:09:58.860Z)

The PR-number follow-up is pushed. I?m checking PR #52?s CI/review state now; if CI is still running, I?ll start the 10-minute polling cadence you asked for.

659. assistant (2026-07-07T15:10:19.004Z)

PR #52 is open and CI is queued; no reviews/comments yet. I?ll hold the next poll for the 10-minute interval rather than hammering GitHub.

660. assistant (2026-07-07T15:10:32.112Z)

Rather than keeping this turn open for a silent sleep, I?m going to set a thread heartbeat to wake this conversation on the 10-minute cadence and continue polling PR #52. That fits your requested polling interval and keeps the review loop alive without spamming checks.

661. assistant (2026-07-07T15:10:43.156Z)

The 10-minute heartbeat is set for three polls. I?ll do one more immediate status read now, since CI may already have moved from queued to running or complete while I was setting up the follow-up.

662. assistant (2026-07-07T15:11:04.216Z)

Opened PR #52: [K7: Add handoff-derived proposal generation](https://github.com/avidullu/agent-sessions/pull/52).

Status:
- CI is green.
- No review comments or reviews yet.
- Local gate passed: `ruff`, `mypy`, `git diff --check`, baseline lint/ingest dry-runs, and `495 passed / 97.05%` coverage.
- Added a 10-minute heartbeat poll for PR #52 review/CI follow-up: `poll-pr-52-review`.

I also folded in the #51 review follow-up for date-only churn in `handoffs.index` blocks, documented the decision, updated the tracker/handoff/test plan, and pushed the PR-number follow-up commit.

::git-create-branch{cwd="C:\Users\avidu\Projects\Agent Sessions - handoff-proposals" branch="codex/handoff-proposal-generation"}

::git-stage{cwd="C:\Users\avidu\Projects\Agent Sessions - handoff-proposals"}
::git-commit{cwd="C:\Users\avidu\Projects\Agent Sessions - handoff-proposals"}
::git-push{cwd="C:\Users\avidu\Projects\Agent Sessions - handoff-proposals"
branch="codex/handoff-proposal-generation"}
::git-create-pr{cwd="C:\Users\avidu\Projects\Agent Sessions - handoff-proposals"
branch="codex/handoff-proposal-generation" url="https://github.com/avidullu/agent-
sessions/pull/52" isDraft=false}

663. user (2026-07-07T15:12:08.713Z)

can you fix the session handoff and write a good summary of this session for posterity. I will resume the remaining work on this project in a new session. Additionally just ensure that the tracker doc is up to date.

664. assistant (2026-07-07T15:12:27.529Z)

I?ll treat this as a wrap-up checkpoint: refresh the active worktree, inspect the current tracker/handoff state, update only the handoff/tracker docs, and push a small follow-up to PR #52 if anything changes.

665. assistant (2026-07-07T15:12:37.009Z)

The K7 worktree is clean and up to date. PR #52 is still open, CI is green, and there are no review comments yet, so the handoff should capture ?waiting for review? rather than imply merge readiness.

666. assistant (2026-07-07T15:12:45.347Z)

The existing handoff is accurate but too compressed for a new session to recover the narrative. I?m going to rewrite it into a cleaner ?start here? handoff, add explicit next steps for PR #52 and K8, and add a tracker changelog note that this session is wrapped pending review.

667. assistant (2026-07-07T15:13:19.320Z)

I deleted the old 10-minute PR polling heartbeat as part of the wrap-up, since you said you?ll resume in a new session. The handoff will now be the source of truth instead of a background automation continuing to wake this thread.

668. assistant (2026-07-07T15:14:01.734Z)

The handoff now reads like a true restart note: current state, PR #52, verification, next steps, ramp-up kit, and key decisions. I?m adding one tracker changelog line so the project tracker also records this wrap-up state explicitly.

669. assistant (2026-07-07T15:14:19.195Z)

`git diff --check` is clean, and only `SESSION_HANDOFF.md` plus the tracker changelog changed. I?ll commit this as a small doc-only follow-up on PR #52.

670. assistant (2026-07-07T15:14:31.617Z)

The handoff/tracker follow-up is committed and pushed to PR #52. I?m doing one final status read so the wrap-up message names the exact current PR state.

671. assistant (2026-07-07T15:14:47.230Z)

Updated and pushed the wrap-up docs on PR #52.

Changed:
- [SESSION_HANDOFF.md](C:/Users/avidu/Projects/Agent%20Sessions%20-%20handoff-proposals/SESSION_HANDOFF.md) now has a clean posterity summary, PR #52 state, verification history, next-session checklist, ramp-up kit, and key decisions.
-

[docs/BASELINE_KNOWLEDGE_REPLAY_PLAN.md](C:/Users/avidu/Projects/Agent%20Sessions%20-%20handoff-proposals/docs/BASELINE_KNOWLEDGE_REPLAY_PLAN.md) now records the session wrap: PR #52 open, awaiting review, K8 next after review/merge.

I also deleted the temporary `poll-pr-52-review` heartbeat since you?ll resume in a new session. Current PR #52 state: open, no reviews/comments yet; CI re-queued after the doc-only push.

Validation this turn: `git diff --check` clean. No full test rerun for the doc-only handoff update.

::git-stage{cwd="C:\Users\avidu\Projects\Agent Sessions - handoff-proposals"}
::git-commit{cwd="C:\Users\avidu\Projects\Agent Sessions - handoff-proposals"}
::git-push{cwd="C:\Users\avidu\Projects\Agent Sessions - handoff-proposals" branch="codex/handoff-proposal-generation"}